algorithms

MDPI

Review

Algorithmic Techniques for GPU Scheduling: A Comprehensive Survey

Robert Chab, Fei Li * and Sanjeev Setia

Department of Computer Science, George Mason University, Fairfax, VA 22030, USA; rchab@gmu.edu (R.C.); setia@gmu.edu (S.S.)
* Correspondence: fli4@gmu.edu; Tel.: +1-703-993-1540

Simple Summary

In this survey, we take a deep look at how different methods are used to schedule tasks on GPUs (graphics processing units), which are powerful chips often used in gaming, AI, and scientific computing. We group these scheduling methods based on the techniques they use—ranging from traditional strategies like step-by-step rules and mathematical models to newer approaches that use machine learning. We also compare how well these methods perform in different real-world applications. Our goal is to understand the pros and cons of each approach, what factors affect scheduling decisions, and how to strike a balance between individual user needs and overall system performance. Our findings show that there is no single best solution. The most effective methods usually combine reliable planning with flexible, learning-based techniques, and use ideas from queue management to stay fair. We also highlight the main challenges in managing GPU resources efficiently and offer some ideas on how to overcome them.

Abstract

In this survey, we provide a comprehensive classification of GPU task scheduling approaches, categorized by their underlying algorithmic techniques and evaluation metrics. We examine traditional methods—including greedy algorithms, dynamic programming, and mathematical programming—alongside advanced machine learning techniques integrated into scheduling policies. We also evaluate the performance of these approaches across diverse applications. This work focuses on understanding the trade-offs among various algorithmic techniques, the architectural and job-level factors influencing scheduling decisions, and the balance between user-level and service-level objectives. The analysis shows that no one paradigm dominates; instead, the highest-performing schedulers blend the predictability of formal methods with the adaptability of learning, often moderated by queueing insights for fairness. We also discuss key challenges in optimizing GPU resource management and suggest potential solutions.

Keywords: GPU scheduling; optimization algorithms; scheduling algorithms; GPU cluster management

check for updates

Academic Editor: Charalampos Konstantopoulos

Received: 1 May 2025
Revised: 12 June 2025
Accepted: 21 June 2025
Published: 25 June 2025

Citation: Chab, R.; Li, F.; Setia, S. Algorithmic Techniques for GPU Scheduling: A Comprehensive Survey. *Algorithms* **2025**, *18*, 385. https://doi.org/10.3390/a18070385

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Over the past decade, graphics processing units (GPUs) have matured from niche graphics accelerators into general-purpose, high-throughput computing engines, fundamentally altering how we approach data- and compute-intensive tasks. As their use has

proliferated beyond rendering—spanning scientific simulation, deep learning, and real-time analytics—traditional CPU-focused schedulers have struggled to balance the massive parallelism and unique memory hierarchies of modern GPU workloads.

This survey brings together findings from across the GPU-scheduling literature to map out both established methods and the latest innovations. We review a spectrum of algorithmic strategies starting with classic formulations such as bin-packing and priority-queue approaches, and moving toward contemporary, data-driven schemes that employ machine learning for adaptive decision making. For each class of algorithms, we discuss its impact on key performance metrics (e.g., throughput, latency, and utilization) and identify the workload or cluster environments in which it excels. Our goal is twofold: to provide researchers with a clear picture of the current state-of-the-art and emerging directions in GPU scheduling, and to provide practitioners with practical guidance for tailoring schedulers to their specific data center architectures and workload profiles.

1.1. The History and Evolution of GPUs

Originally conceived for real-time graphics rendering, GPUs have since outgrown their narrow origins to become indispensable engines for a wide spectrum of compute-intensive tasks. Their story begins in 1968 with the founding of Evans & Sutherland Computer Corporation to build custom graphics hardware [1]. A defining moment arrived in 1999 when NVIDIA introduced the GeForce 256—often credited as the first "modern" GPU, which established the benchmark for future GPU designs and capabilities [2].

Over the following decades, GPU architectures have seen dramatic enhancements, most notably in programmability, allowing developers to exploit massive parallelism for non-graphical workloads [3]. Today, GPUs drive applications across gaming, professional visualization, high-performance scientific computing, machine learning, and even cryptocurrency mining [4–6]. The market has diversified into several segments—consumer and gaming GPUs, professional and workstation cards, data-center artificial intelligence (AI) accelerators, and mobile/embedded solutions—each optimized for distinct performance, power, and feature requirements [7–9].

A pivotal milestone arrived in 2006 with NVIDIA's unveiling of the Tesla architecture, which for the first time incorporated unified shader functionality [3]. Shaders—small programs executed on the GPU to manipulate graphics data—had until then been processed through distinct vertex and pixel pipelines [10]. By unifying these pipelines, every core gained the ability to execute any shader type, vastly increasing both flexibility and programmability. This architectural evolution laid the groundwork for general-purpose GPU computing frameworks such as CUDA (compute unified device architecture) [3,11].

In the years since, GPUs have become omnipresent, powering devices from high-end desktop workstations to smartphones [2,12]. High-performance systems typically employ dedicated GPUs, while integrated GPUs (iGPUs), embedded within the CPU and sharing system memory, are widespread in entry- to mid-level consumer hardware [2,13]. Although iGPUs offer advantages in cost and energy efficiency, they lag significantly in raw compute power: for example, a consumer-grade dedicated GPU of a given generation can deliver between four and twenty-three times the single-precision floating-point throughput of its integrated counterpart [13,14].

1.2. GPU vs. CPU: Similarities and Differences in Architecture and Fundamentals

A meaningful comparison between CPUs and GPUs spans architectural design, performance characteristics, memory hierarchy, and resource management. Traditional CPUs prioritize sequential instruction throughput and general-purpose flexibility, while modern GPUs consist of thousands of smaller, specialized cores engineered for massive data paral-

lelism [15–18]. For example, while a high-end server CPU may reach roughly 192 cores, the NVIDIA A100 GPU boasts 6912 cores, dramatically illustrating the divergent scaling strategies of these platforms [19,20]. CPUs often encounter diminishing returns from increase in core count—constrained by Amdahl's law and the overhead of sequential resource management—while GPUs excel in scheduling and running thousands of parallel threads with minimal latency [18,21–24]. Consequently, for highly parallelizable workloads such as scientific simulations and deep learning, GPUs can achieve speedups ranging from 55× to over 100× compared to CPUs, a performance gap that further widens with larger model sizes and batch dimensions [25–27].

Memory architecture further accentuates the contrast between CPUs and GPUs. GPU workloads—particularly in graphics rendering and deep learning—rely on extremely high memory bandwidth to support their massive parallel operations [28]. To meet these demands, GPUs employ specialized on-board memory (e.g., VRAM) and allocate a much larger proportion of registers relative to main memory than CPUs do [29,30]. Consequently, modern data-center GPUs can deliver up to 54 × the memory bandwidth of comparable CPUs, highlighting a profound disparity in data-movement capability [27].

Resource management also diverges sharply between the two architectures. In standalone systems, the GPU driver orchestrates memory allocation, context switching, and command handling, serving as the critical interface between the operating system and the hardware [31,32]. Meanwhile, runtime frameworks such as CUDA and OpenCL furnish low-level scheduling and execution controls [33,34]. At scale, high-performance computing clusters layer in sophisticated orchestration platforms—SLURM, Borg, Yarn, and Kubernetes—to allocate GPU resources, track utilization, and optimize throughput across nodes [35–39]. These systems enforce workload isolation, priority policies, and cluster-wide performance tuning [40], yet the GPU driver remains indispensable for fine-grained, device-level control of the hardware [41].

The performance of GPUs is typically evaluated across several key metrics, as outlined below. *Floating-point operations per second* (FLOPs) measure raw arithmetic throughput and are broken out by precision—16-bit (half), 32-bit (single), 64-bit (double), and, more recently, 8-bit operations [42–45]. *Tensor performance*, reported in TOPS (tera operations per second), reflects the capabilities of specialized tensor cores for AI/ML (machine learning) workloads (e.g., INT8 TOPS for inference or FP16 TFLOPS for training) [46]. *Memory bandwidth*, in GB/s or TB/s, gauges how quickly data can be moved on and off the GPU—an essential factor for large-scale deep learning [45]. The *number of CUDA cores* (Nvidia) or stream processors (AMD) indicates parallelism potential [19,47], while *VRAM capacity* determines the size of models and datasets that can be held in memory [45]. Additionally, *clock speed* (MHz/GHz) impacts per-core instruction throughput [48], *PCIe (peripheral component interconnect express) bandwidth* governs CPU-GPU data transfer rates [49], and *power consumption* (Watts) affects overall energy efficiency and cooling requirements [45,48].

Beyond raw numbers, a GPU's compute capability defines its supported feature set—such as specific CUDA compute capability levels on Nvidia hardware [45]. Modern data-center GPUs like the Nvidia A100 illustrate these principles in practice: they pack 6912 cores, 40 GB or 80 GB of VRAM, and 54.2 billion transistors, delivering up to 624 TFLOPS in half-precision or 312 TFLOPS in single-precision workloads [42–44]. As machine learning (ML) demands continue to grow, architectures such as the A100 are specifically engineered to optimize both scalability and efficiency for large-scale AI training and inference [1,50].

1.3. Cluster Computing and the Necessity of GPU Scheduling Algorithms

Cluster computing has undergone significant evolution over the past few decades. The initial drive for greater computational throughput gave rise to CPU clusters composed

of a small number of high-performance cores [51]. However, as machine learning and deep learning (DL) workloads surged, GPUs have become an indispensable element of modern infrastructure. Large-scale GPU clusters—once the domain of specialized scientific centers—are now commonplace in academic, industrial, and cloud environments [52–54]. These systems comprise interconnected nodes, each hosting one or more GPUs, and often feature heterogeneous hardware configurations that integrate diverse CPU architectures, memory hierarchies, and GPU generations [44,55–57]. To support the high data rates demanded by ML/DL workloads, GPU clusters employ high-speed interconnects—such as PCIe, NVLink, and InfiniBand—to facilitate rapid data exchange both within and across nodes [58,59]. Figure 1 depicts a representative GPU cluster architecture.

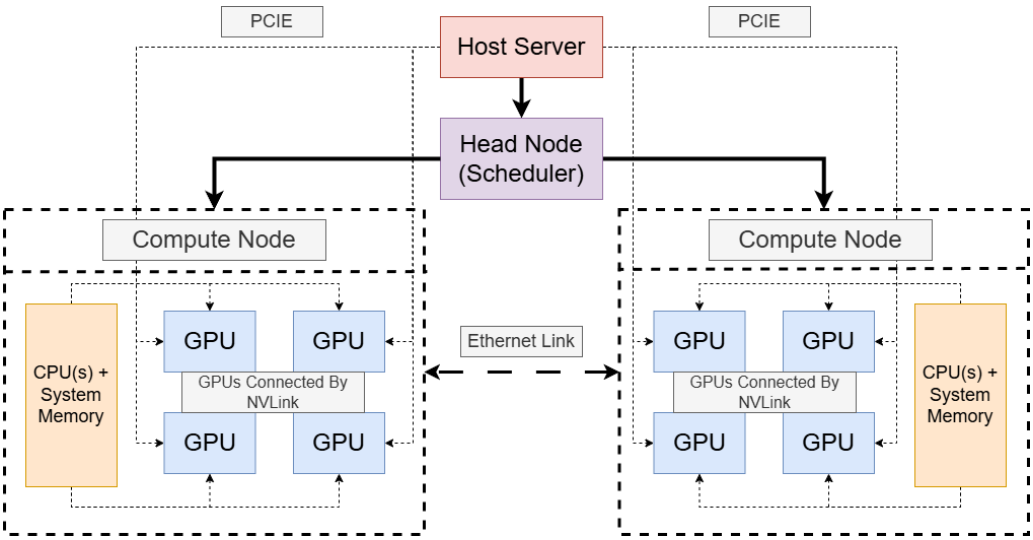

Figure 1. The general setup of a GPU cluster: The dashed arrows represent data-flow, and the solid arrows represent scheduling decisions.

In modern GPU clusters, the high capital and operational costs of GPU hardware and supporting infrastructure demand sophisticated scheduling strategies. High-end GPUs such as the Nvidia A100 80 GB can cost nearly $15,000 [60], and utilization rates in large-scale deployments can plummet to 50 % [44,61]. Such under-utilization reflects not a shortage of demand but suboptimal resource allocation. Critical resources—including GPU and CPU compute units, memory bandwidth, and interconnect bandwidth—are both scarce and expensive [36,56,62,63]. Advanced scheduling algorithms that optimize across these dimensions are therefore essential to minimize idle time, balance operator and user objectives, and achieve substantial cost efficiencies in large-scale environments.

Several intertwined factors exacerbate scheduling challenges in these clusters. Heterogeneity extends beyond hardware to encompass a wide spectrum of job profiles: from training workloads that run for days [64] to latency-sensitive inference tasks that complete in milliseconds [65]. Uncertainty in execution times further complicates allocation decisions, as does the diversity of resource requests—ranging from fractional GPUs to multi-GPU or CPU-only configurations. Consequently, simple heuristics such as first-fit or bin-packing fail to capture the multi-dimensional nature of GPU cluster scheduling [36].

Scheduling challenges are also exacerbated by conflicting objectives between cluster operators and users. Operators typically aim to optimize GPU and resource utilization while minimizing job completion times [56,66–68], while users prioritize job accuracy and fairness in allocation [62,69,70]. As a result, no single scheduling solution is universally applicable. Algorithmic scheduling techniques have already shown impressive performance, with one algorithm reducing unallocated GPUs by up to 49% [36]. Such improvements could result in significant cost savings, particularly in large-scale data centers with thou-

sands of GPUs. For instance, Helios, operated by SenseTime, manages four clusters with a total of 6416 GPUs [64].

Building on the challenges outlined above, LLMs (large language models) further amplify the scale and complexity of GPU cluster scheduling. In certain LLM configurations, GPU utilization can dip below 50%, and decreases further as model parameter counts grow—despite proposed methods that can boost utilization to over 75% [61]. Resource allocation for LLM training brings additional difficulties not encountered in standard deep learning workloads, since most existing schedulers were not designed to handle the vast scale, heterogeneity, and dynamic demands of LLMs. Overcoming these challenges will require more sophisticated scheduling strategies.

1.4. Survey Methodology

A rigorous, multi–stage screening pipeline ensured that the survey is systematic rather than anecdotal. Figure 2 summarizes the workflow; the text below details each decision point and the approximate record counts.

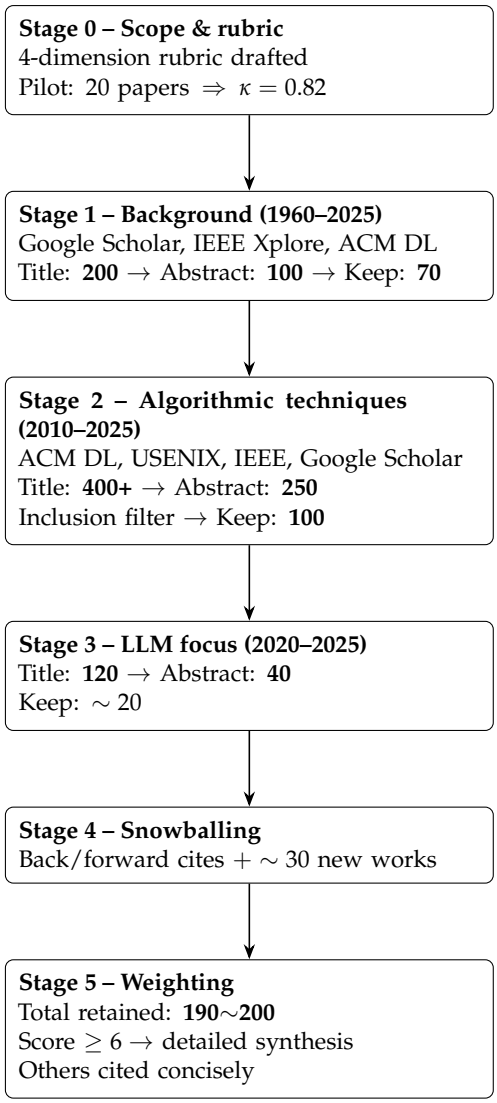

Figure 2. Multi-stage screening pipeline. Counts rounded to nearest 10.

Stage 0: scope definition and rubric design. Before any database queries, we drafted the four—dimension quality—weighting rubric in Table 1. Pilot screening on a conve-

nience sample of 20 papers confirmed that the rubric discriminated well between position statements, theoretical work, and large-scale empirical studies.

Table 1. Quality-weighting rubric used to guide the depth of coverage (maximum possible score: 8).

Dimension Evidence Required	Score Range	
Strong theoretical guarantees	0–2	Formal bounds or proofs (e.g., competitive ratio).
Hardware realism	0–2	Evaluation on clusters with at least 8 GPUs or production-scale traces.
Comparative analysis	0–2	Benchmarking against standard baselines.
Modern DL relevance	0–2	Targets large-scale deep learning or LLM workloads.

Stage 1: background and systems context (1960–2025). Databases searched: Google Scholar, IEEE Xplore, ACM DL, manufacturer websites for official specifications. Query families:

`‘‘GPU AND history’’, ‘‘CPU-GPU architectural comparison’’, ‘‘cluster topology evolution’’, ‘‘GPU architecture’’.`

Roughly 200 candidate records were identified by title screening; abstract screening reduced this to ≈100. After full-text reading we retained 70 sources that (i) provided quantitative historical context or (ii) framed scheduling problems from a systems perspective. These sources underpin the Introduction.

Stage 2: algorithmic techniques (2010–2025). Databases searched: ACM DL, USENIX, IEEE Xplore, Google Scholar. Canonical query:

`‘‘GPU’’ AND ‘‘cluster’’ AND (scheduler OR scheduling) AND (coflow OR multi-objective OR reinforcement OR prediction OR heuristic OR energy-aware OR techniques OR metaheuristic).`

Title screening surfaced more than 400 unique records. Abstract screening—applying the inclusion criteria listed below—reduced the pool to ≈250 (ten papers overlapped with Stage 1 and were carried forward). Full-text review against the rubric yielded roughly 100 high-quality papers, weighted in proportion to their rubric scores when allocating discussion space in Sections 2 and 3.

A study was included if it (i) addressed the scheduling of multiple GPUs, (ii) tackled heterogeneous hardware, multi-objective optimization, prediction-enhanced policies, or formal guarantees, and (iii) provided empirical evidence or rigorous theory. Position papers without such support and single-GPU runtime optimizations were excluded.

Stage 3: LLM-specific scheduling (2020–2025). Given the rapid rise of large language models (LLMs), we executed a dedicated search phase focused on inference/training at scale. Query example:

`‘‘LLM inference’’ AND ‘‘GPU scheduling’’ AND coflow.`

Given the rapid rise of large language models, we executed a dedicated search phase focused on inference/training at scale. **Query example:** `‘‘LLM inference’’ AND ‘‘GPU scheduling’’ AND coflow`. Approximately 120 titles were screened; abstract review left ≈ 40. Full-text assessment against the Stage 2 criteria produced a final set of about 20 papers, complemented by Stage 2 studies that offered immediately applicable techniques.

Stage 4: citation snowballing and gap filling. Backward and forward snowballing on the highest-weighted papers unearthed about 30 additional works, often preprints that had not yet appeared in indexed databases. Where these filled clear topical gaps (e.g., secure multi-tenant isolation), they were assimilated and scored.

Stage 5: weighting and depth of coverage. All 220∼240 retained studies were scored on the rubric in Table 1. Papers that scored more than 6 or were truly excellent in a specific category that served as their clear focus receive detailed narrative synthesis and appear in comparative studies in Section 3; lower-scoring papers are cited for completeness but discussed more briefly.

Editorial coverage policy. Rubric score determined eligibility for detailed synthesis, but not every high–scoring study could receive equal space. When several papers shared an identical algorithmic core (e.g., three LAS–based queueing heuristics with comparable results) we selected one or two representative exemplars for in-depth narrative and tabular comparison; the remaining high–scoring papers are cited concisely in the same subsection. This preserves coverage breadth while avoiding redundant exposition.

1.5. Limitations of Classical Assumptions

Classical queueing abstractions—most notably the $M/G/k$ model—underpin much of the early literature on GPU and cluster scheduling. However, these models typically assume light-tailed service time distributions, often exponential. In contrast, empirical traces from modern deep learning (DL) clusters reveal markedly heavier tails:

- Alibaba GPU Trace (2021): the 90th percentile job duration is 3.5× the mean;
- Google Borg Trace (2019): the 90th percentile job duration is approximately 3× the mean.

This discrepancy is consequential: schedulers built on light-tailed assumptions systematically under-provision for queue-length spikes and tail-latency outliers. In heavy-tailed regimes, even a few large jobs can monopolize resources for extended periods, significantly delaying moderate-sized jobs—an effect that exponential models fail to capture. Any queueing-theoretic analysis or simulation framework must therefore incorporate heavy-tail parameter estimates to remain predictive at scale.

To bridge the gap between theoretical models and real-world DL system behavior, we annotate each optimization paradigm in the following sections with a brief *design takeaway*. These highlight where the paradigm's assumptions diverge from production realities—e.g., in memory locality, co-flow bottlenecks, or preemption latencies—and help identify when classical insights may need revision or augmentation.

1.6. Paper Overview

This survey is structured as follows. Section 2 reviews a range of scheduling models and algorithmic paradigms, grouping them into heuristic-based, optimization-based, learning-driven, and queuing-theoretic approaches. Section 3 offers a comparative analysis of these methods, highlighting their respective strengths and weaknesses, the objectives they target, and practical considerations for real-world deployment. Section 4 discusses the challenges posed by LLM workloads and details how current techniques must be expanded to overcome these obstacles. Finally, Section 5 identifies open challenges in GPU resource management and proposes promising directions for future research.

Throughout the paper, we introduce GPU-specific terminology as it becomes relevant. For standard scheduling concepts not covered here, we refer the reader to Pinedo's comprehensive treatment [71]. A consolidated list of abbreviations is provided in the Abbreviations table for quick reference.

2. Models

Accurate modeling is the foundation of modern GPU cluster scheduling, allowing us to capture intricate real-world constraints and objectives. A formal model provides a principled framework for reasoning about resource allocation, assessing trade-offs, and guaranteeing desired service levels. Without such models, simplistic schedulers may leave GPUs idle, violate fairness criteria, or fail to meet QoS (quality of service) and SLO (service level objectives) targets. For instance, a cluster hosting long-running ML training workloads must allocate GPUs judiciously to prevent both idle hardware and excessive queuing delays [43,72].

In this section, we survey the key modeling techniques used in GPU cluster scheduling. We begin by identifying the core components present in most models, then present a taxonomy of modeling approaches. Afterward, we review the metrics commonly employed to evaluate scheduler performance, and conclude with a comparative discussion of each approach's strengths and weaknesses.

By casting objectives—such as minimizing job completion time or maximizing throughput—into precise mathematical formulations, these models directly inform the design of efficient scheduling algorithms.

2.1. Components of a Model

Most scheduling theory is *hardware-agnostic*, but GPU clusters differ significantly from traditional CPU farms. Table 2 highlights key architectural traits that strongly influence scheduling algorithm design. Frequent preemption—central to SRPT (shortest-remaining-processing-time)-style CPU scheduling—is generally impractical on GPUs. Interconnect contention necessitates topology-aware bin-packing or mixed-integer linear programming (MILP) approaches. NVIDIA's Multi-Instance GPU (MIG) allows resource multiplexing, but only at coarse, integer granularity. Accurate power modeling must decouple streaming multiprocessor (SM) and high bandwidth memory (HBM) usage, in contrast to the unified RAPL (running average power limit)-based models used for CPUs.

Table 2. Hardware assumptions and their scheduling implications.

Trait	Typical CPU Cluster	GPU Cluster (A100/H100 era)
Context switch / preemption	≈ 3 µ∼10 µs (core) inexpensive, fine-grained	10 ms∼30 ms (kernel) costly; whole-SM drain required [73,74]
Device memory per worker	128, 256 GB DDR4/5 shared	40 $\sim$ 80 GB HBM private to each device Placement must avoid OOM even at low utilization.
NUMA / socket locality	1∼4 hop latency tiers per node [75]	Two tiers: NVLink/NVSwitch (200∼900 GB/s) vs. PCIe (32 GB/s, 64 GB/s). Cross-tier traffic quickly dominates runtime [58,76].
Partitioning granularity	Core / SMT thread; OS-level control groups	MIG slices (1/7∼1/2 device) or whole GPU Integer-knapsack bin packing, no fractional share scheduling.
Inter-device topology	NUMA DRAM buses and Ethernet/IB	On-node all-to-all (NVSwitch) + inter-node fat-tree IB (NDR/HDR) Topology-aware placement yields up to 1.3× speed-ups [77].
Power management	Per-core DVFS, RAPL capping	Device-level DVFS; memory clock often fixed Energy models must separate SM vs. HBM power.

GPU cluster architecture: Models typically represent the cluster as a set of compute nodes, each equipped with one or more GPUs, along with CPUs, memory, and high-speed

network interfaces. Within a node, GPUs communicate over a fast interconnect, while nodes themselves are linked by a high-speed network, enabling workloads to span multiple machines [78,79]. A *central scheduler* or *cluster manager* receives user submissions and determines how to assign GPUs to each job [80]. Jobs may demand fractions of a GPU, a single GPU, or multiple GPUs. Unless stated otherwise, we assume jobs arrive online and wait in a queue until sufficient resources become available.

Jobs and GPU requirements: We model three primary entities: *jobs*, *resources*, and *queues*. Let $J = \{1, 2, \ldots, n\}$ denote the set of jobs to be scheduled. Each job $j \in J$ is characterized by:

- GPU demand g_j (which may be fractional or span multiple GPUs),
- estimated runtime p_j,
- release time r_j (for online arrivals),
- optional priority or weight.

The cluster provides m GPU slots, which may be homogeneous or heterogeneous across nodes. Scheduling systems may use a single global queue or multiple queues (e.g., to separate jobs by GPU type or priority) [81]. Some models allow preemption, while others assume jobs run non-preemptively to completion [82,83]. In offline formulations, job runtimes may be known or stochastic; in online settings, runtimes are unknown and decisions must be made with incomplete information [74,84]. Gang scheduling is often assumed for multi-GPU jobs, meaning a job requiring g_j GPUs only starts when all g_j GPUs are free, and those GPUs remain dedicated until the job finishes [44,85–87]. Some models ignore inter-GPU communication overhead, whereas others explicitly optimize job placement to minimize it [88,89]. Finally, models differ in whether they assume cluster homogeneity or account for mixed GPU types and generations [44,55].

Scheduling timeline: Scheduling models vary in their representation of time, adopting either discrete or continuous frameworks. Discrete-time formulations divide the schedule into uniform slots, which are well suited to Markov decision processes and dynamic programming approaches [82]. Continuous-time models, by contrast, treat job arrivals and completions as asynchronous events, capturing finer temporal dynamics. Many models further simplify by assuming non-preemptive execution, where jobs run to completion once scheduled. Preemptive models—used for GPU time-sharing or when supporting checkpoint–and–restart—must quantify preemption overheads [90]. Finally, time-sharing frameworks introduce a *time quantum*, the fixed interval for which GPUs are leased to jobs before the scheduler reevaluates resource allocations [91].

Objectives: Let the system state be defined by the set of pending jobs and available GPUs. A scheduling model then specifies an objective function to optimize and a set of constraints to enforce. At a high level, the scheduler must decide which jobs to run, when, and on which GPUs to optimize metrics such as completion time or throughput. Formally, this becomes a combinatorial optimization problem: for example, binary variables $x_{j,k,t} \in \{0, 1\}$ can indicate whether job j starts on GPU k at time t. Constraints ensure each GPU runs at most one job at a time, that job j occupies g_j GPUs for its processing time p_j, and that allocations respect job demands and hardware limits (e.g., preventing over-allocation under multi-instance virtualization). Even simple objectives—such as minimizing makespan $C_{\max}$ on m identical machines (problem $P \parallel C_{\max}$)—are NP-hard for $m \geq 2$ [92]. Similarly, minimizing total completion time $\sum_j C_j$ or average waiting time is NP-hard [93,94]. Consequently, exact optimization is intractable at scale, and most models instead provide a formal foundation for developing and analyzing heuristics and approximation algorithms [95]. A summary of common scheduling objectives appears in Table 3.

Table 3. Scheduling objectives: formulations and definitions.

Objective	Formulation	Definition
Minimizing average waiting time	$\min \frac{1}{n} \sum_{j=1}^{n} W_j$	Shorten the average time jobs wait in the queue before execution.
Minimizing average completion time	$\min \frac{1}{n} \sum_{j=1}^{n} C_j$	Minimize the average turnaround time between job submission and completion.
Maximizing throughput	$\max \frac{\#\text{jobs finished}}{\text{time}}$	Increase the number of jobs completed per unit time.
Maximizing utilization	$\max \frac{\text{total busy time}}{\text{total available time}}$	Maximize the fraction of total time during which GPUs are actively utilized.
Maximizing fairness	$\min \max_j \left(\frac{C_j}{T_j} \right)$	Equalize resource allocation by preventing any job j from experiencing excessive slowdown relative to its service time T_j.
Minimizing energy consumption	$\min \sum_{j=1}^{n} E_j$	Reduce the total energy consumed by the GPU cluster during job scheduling.

Note: Each expression is minimized (or maximized) over the feasible schedule set $\mathcal{S}$.

Beyond the core objectives defined in Table 3, scheduling models often refine or combine multiple performance metrics. Many objectives are functions of a job's completion time C_j, or related measures such as waiting time $W_j = C_j - r_j - p_j$. Common examples include minimizing average waiting time, or maximizing throughput, i.e., the number of jobs completed per unit time [96]. Alternative objectives target the makespan $\max_j C_j$, deadline compliance, or fairness metrics like slowdown—the ratio of a job's response time to its processing time—to prevent excessive delays for any job [81,97].

Fairness is a central concern in many models. Schedulers may minimize the maximum slowdown across all jobs or ensure that each job's performance is at least as good as if it ran in isolation on an equal share of the cluster. Mechanisms such as dominant resource fairness (DRF) and its variants (e.g., implemented in Dorm) aim to allocate the dominant resource—GPUs—equitably among users over time [98,99].

Energy efficiency is another critical objective. Some formulations incorporate constraints or multi-objective functions that trade-off performance and power consumption. Techniques like dynamic voltage and frequency scaling (DVFS) or powering down idle GPUs can be modeled to minimize total energy usage [100–104]. Energy-aware schedulers seek to reduce GPU power draw while still meeting performance SLOs [105–107].

In clusters dominated by ML workloads, scheduling metrics often extend beyond runtime. For example, SLAQ (SLA-aware queuing scheduler) evaluates performance based on the final model accuracy or training loss, rather than just job duration. Other models incorporate job-specific deadlines or accuracy targets directly into the objective function [108].

In summary, a rigorous scheduling model precisely defines (1) what constitutes a feasible schedule—through resource and timing constraints—and (2) what constitutes a good schedule—through one or more explicitly stated objective functions. This formalism underpins the systematic design and analysis of effective scheduling algorithms.

Design takeaways. Each scheduling objective entails trade-offs that directly impact real-world performance. For example, minimizing average latency can degrade tail-latency

SLOs, particularly under heavy-tailed job-size distributions—a common pattern in DL training workloads. Similarly, maximizing GPU-hour throughput may lead to starvation of short, interactive inference jobs unless fairness constraints are explicitly enforced. In multi-tenant clusters, optimizing solely for utilization can exacerbate interference between jobs, resulting in volatile 95th-percentile latencies. As a result, effective scheduling demands a careful balance between latency, throughput, and fairness. Many modern schedulers address this by adopting weighted or multi-objective formulations that explicitly navigate these competing goals.

2.2. Categorization of Models

GPU scheduling problems can be modeled in various ways, each capturing different assumptions about job arrivals, system information, and analysis techniques. We categorize these models along several key dimensions and highlight their respective strengths and weaknesses.

Offline and online scheduling models: A fundamental distinction is whether all jobs are known in advance (*offline scheduling*) or arrive over time (*online scheduling*). Offline models assume a fixed set of jobs $J = \{1, 2, \ldots, n\}$ with known runtimes and resource requirements. The goal is to compute a schedule—an assignment of start times and GPUs to each job—that optimizes a chosen objective (e.g., minimizing total completion time or the makespan). Offline scheduling can be formulated as a deterministic optimization problem, often reducing to NP-hard variants of classic machine scheduling problems [109]. With complete information, near-optimal or optimal schedules can be obtained for moderate-sized instances via integer programming or exhaustive search; however, these approaches do not scale to the large, dynamic workloads observed in real GPU clusters [110].

In contrast, online models treat scheduling as an ongoing process in which jobs arrive according to some distributions and decisions are made without knowledge of future arrivals. The scheduler reacts in real time to job arrivals and completions. Online algorithms are commonly evaluated via *competitive analysis*, proving performance bounds relative to an offline optimum—for example, guaranteeing a completion time at most c times the optimal (*c-competitive*) [111,112]. Strong worst-case guarantees are often elusive for complex objectives, and without resource augmentation the competitive ratio may be unbounded. To obtain more meaningful metrics, many analyses instead assume stochastic workloads (e.g., Poisson arrivals with rate λ, job sizes drawn from known distributions), which enables optimization of expected performance or derivation of steady-state results via queueing theory [113]. Classical queueing models such as the $M/G/k$ system (Poisson arrivals, general service times, k servers) yield predictions for waiting times and queue lengths, and form the basis for many practical online GPU schedulers that serve jobs submitted unpredictably by users [74]. Some recent models further incorporate limited lookahead or predictions—using, for instance, user-provided runtime estimates or historical profiles—to blend online reactivity with partial future knowledge [114].

Design takeaways. Offline scheduling assumes complete knowledge of future job arrivals and service requirements—an assumption that rarely holds in real-world GPU clusters. In practice, even short look-ahead windows (e.g., tens of seconds) struggle to predict deep learning job runtimes accurately, due to variability in data pipelines and dynamic resource contention. Consequently, algorithms optimized for offline settings often overfit to idealized conditions and may perform poorly under unpredictable, real-time workloads. In production, practitioners typically rely on hybrid online heuristics that incorporate short-term forecasts or runtime profiling, rather than purely offline strategies.

Queueing-theory-based models: Queueing-theoretic abstractions treat each GPU or collection of GPUs as a server and model jobs as customers, yielding analytic expressions

for performance metrics such as mean waiting time or slowdown. Classical results underpin several scheduling policies: when job sizes are known, the shortest remaining processing time (SRPT) discipline minimizes mean completion time; when sizes are unknown, the Gittins index policy achieves optimal mean slowdown. Likewise, least attained service (LAS)—implemented in Tiresias via a time-slicing scheme—draws directly on these theoretical insights and operates effectively without explicit size estimates.

At the scale of multi-GPU clusters, systems are often represented as an $M/G/k$ queue with a central dispatcher [85]. Such models, however, typically abstract away critical factors like placement constraints and inter-GPU communication overheads. In real-world setups, jobs may require data locality or specialized hardware features (e.g., tensor cores) and can suffer performance degradation if spread across distant nodes –issues that a simple $M/G/k$ framework cannot capture.

To address these limitations, more sophisticated queueing formulations introduce multi-class networks: separate queues represent different GPU types or workload classes, and additional parameters model network delays in distributed training or inference. While these extensions better capture heterogeneity and locality, they trade off analytical tractability. Nonetheless, even simplified queueing approaches remain valuable for informing scheduler design—for instance, by quantifying the impact of FCFS versus priority-based service disciplines on average waiting times under varying load conditions [115].

Design takeaways. Classical queueing models such as $M/G/k$ assume Poisson arrivals and light-tailed or exponential service time distributions—assumptions that do not hold for deep learning workloads. Empirical traces, such as Alibaba-GPU-2021 (90th percentile $\approx 3.5\times$ mean) and Google Borg (90th percentile $\approx 3\times$ mean), reveal heavy-tailed job durations. Fitting an exponential distribution to such data leads to systematic underestimation of tail latency. In particular, mis-specifying an $M/G/k$ model under realistic load conditions (e.g., utilization $\rho \approx 0.8$) can significantly inflate p95 response times. As a result, when schedulers rely on queueing-theoretic insights, they should either incorporate heavy-tail parameter estimates or fall back to trace-driven simulations to avoid overly optimistic performance projections.

Optimization-based models: A prevalent approach formulates GPU scheduling as a mixed-integer optimization problem. In time-indexed MILP formulations, binary variables $x_{j,k,t}$ indicate whether job j runs on GPU k at time t; alternative formulations introduce assignment variables $x_{j,k}$ and ordering variables $y_{j,j'}$ to capture both placement and sequencing decisions [116,117]. Common objectives include minimizing total (weighted) completion time or maximizing fairness, subject to constraints that enforce at most one job per GPU at any time and any job precedence relations. While these models afford exactness, their variable count grows with the product of jobs, GPUs, and time slots, making them computationally expensive for large instances. Consequently, MILP-based methods are most appropriate in offline settings for small to medium clusters or as benchmarks for heuristic algorithms, where modest problem sizes can often be solved to optimality [95].

To extend optimization into dynamic environments, some systems employ a rolling-horizon strategy: they repeatedly solve an MILP over the current queue or a short future window, then enact the resulting allocation before re-optimizing as new jobs arrive. For example, TetriSched (not GPU-specific) uses an MILP to allocate resources based on learned inference models [110], and Gavel recasts various scheduling policies into optimization problems tailored for heterogeneous clusters [57]. Alternatively, hybrid frameworks like Cynthia formulate a cost-minimization problem with performance guarantees and then apply a bounded-search heuristic guided by lightweight runtime predictions to efficiently discover near-optimal resource configurations in cloud-based DDNN (distributed deep neural network) provisioning [118].

Design takeaways. While pure MILP formulations offer exact solutions, their variable count scales with |jobs| × |GPUs| × |time slots|, rendering them impractical for large-scale or highly dynamic clusters. In offline settings on small to medium clusters, MILP can yield optimal schedules, but even rolling-horizon variants incur significant solver latency—ranging from tens of seconds to minutes—and may struggle to adapt when high-priority DL jobs arrive unexpectedly. Moreover, cost models (e.g., learned runtime predictions) are often noisy; optimization routines that depend on misestimated job characteristics can under-perform relative to simpler heuristics, especially under bursty workloads. In practice, MILP is most effective as a benchmarking tool or within tightly controlled batch environments, while latency-sensitive clusters benefit more from fast approximations or hybrid heuristic approaches.

Optimization-inspired heuristics: Production GPU schedulers often employ simple, fast heuristics that support real-time decision making and ease of deployment [79]. A canonical example is a shortest-remaining-processing-time (SRPT) first greedy policy, which assigns the smallest pending job to any free GPU immediately upon becoming available. While such rules do not guarantee global optimality, they offer high responsiveness and lend themselves to probabilistic analysis. For instance, modeling GPUs as bins and arriving jobs as items enables evaluation of first-fit and best-fit decreasing placement heuristics:

1. First-fit: assign each job to the first available GPU or time slot.
2. Best-fit: assign each job to the GPU or slot that minimizes leftover capacity.

Despite their simplicity, these heuristics draw on combinatorial optimization principles and are straightforward to simulate and analyze.

Backfilling is a more advanced heuristic that boosts utilization by opportunistically scheduling smaller jobs ahead of larger, blocked jobs [119]. When the head-of-line job requires multiple GPUs and must wait, backfilling fills idle resources with shorter jobs, thereby reducing overall idle time. This approach is ubiquitous in HPC schedulers [120,121] and has been effectively adapted for GPU clusters.

Dynamic programming (DP) can precisely solve certain simplified scheduling problems using state-space recurrences, where each state represents the set of completed and running jobs. However, the exponential growth of the state space makes DP infeasible at cluster scale. The heterogeneous earliest-finish-time (HEFT) algorithm provides a practical alternative: it leverages DP-inspired analysis on a DAG of tasks, prioritizing and mapping jobs onto heterogeneous processors based on estimated finish times. This approach yields near-optimal schedules without the need for exhaustive enumeration [122,123].

Hybrid methods combine optimization models with efficient combinatorial algorithms to balance solution quality and scalability. Allox, for example, constructs a bipartite graph at each scheduling interval—pending jobs on one side, available resources on the other—with edge weights reflecting estimated completion times, and solves a minimum-cost matching to assign jobs so as to minimize overall delay [124]. By leveraging specialized matching algorithms rather than brute-force searches, these hybrid heuristics achieve practical performance at scale.

Design takeaways. Greedy policies (e.g., shortest-job-first, first-fit, best-fit) and backfilling excel at minimizing overhead and enabling near-real-time decision-making. However, they can lead to starvation of large DL training jobs or violate fairness in multi-tenant environments. While backfilling boosts GPU utilization by filling idle GPUs with shorter jobs, it can also cause head-of-line blocking for critical workloads if reservation strategies are not carefully designed. DP-inspired approaches like HEFT are effective for DAG-structured tasks but do not generalize well to arbitrary job graphs or mixed training/inference workloads. Hybrid matching-based methods (e.g., bipartite matching in Allox) strike a balance, achieving high GPU utilization while limiting solver time, but they still rely on accurate task

duration estimates. In production, teams often augment these heuristics with lightweight prediction modules (e.g., runtime predictors or simple interference models) to mitigate misallocation caused by DL-specific variability.

Learning-based models: Recent scheduling frameworks leverage machine learning to predict job characteristics or adaptively optimize decisions. These learning-based models are typically classified into three categories: *ML-assisted prediction models*, *reinforcement learning (RL) models*, and *hybrid learning models*.

ML-assisted prediction models employ learned estimators to predict key job parameters—such as runtime or resource requirements—before scheduling. For each job j, a predictor $\hat{p}_j = f(\text{job features})$ (e.g., a linear regression, a neural network trained on historical logs, or user-provided estimates) approximates the true value p_j [125]. By incorporating $\hat{p}_j$ into placement and ordering decisions, schedulers can significantly improve backfilling efficiency and reduce queueing delays in HPC clusters [126]. For example, Optimus probes performance across varying GPU counts to build an online throughput model, dynamically allocating GPUs to maximize aggregate throughput [66], while AlloX performs brief profiling runs on CPU and GPU, fits a linear per-iteration time model, and selects the optimal platform for each job [124]. Robust implementations must also include mechanisms to tolerate or adapt to prediction errors.

RL models frame scheduling as a Markov decision process (MDP), where states encode resource utilization and queue lengths, actions correspond to dispatch or prioritization decisions, and rewards reflect metrics such as throughput or slowdown. An RL agent learns a dispatch policy by interacting with a simulated cluster or replaying historical traces. DeepRM, for instance, embeds both current and queued jobs in its state representation and dispatches batches to minimize average slowdown [82], while Decima incorporates a graph neural network to capture DAG-structured workloads and learns execution orderings that minimize end-to-end runtime [127]. These models can balance multi-objective goals—efficiency, fairness, QoS—through reward shaping, but their performance often hinges on the fidelity of the training environment. Recent multi-agent RL approaches extend this paradigm by treating GPUs or individual jobs as agents that coordinate to improve global performance [128].

Hybrid learning models combine the adaptability of ML with the predictability of classical heuristics. In these layered systems, ML components tune critical heuristic parameters—such as time-slicing intervals—while the final scheduling decisions rely on traditional rule-based mechanisms. Themis, for example, augments an auction-based scheduler by optimizing bid values through ML-driven models [85], and Pollux continuously adjusts per-job resource allocations (batch size and GPU count) based on observed throughput to steer training workloads toward optimal efficiency [68].

Design takeaways. ML-assisted prediction models can significantly reduce queuing delays by providing schedulers with accurate estimates of job runtime or throughput. However, their benefits diminish rapidly as prediction errors increase. In production DL clusters, factors such as data pipeline skew or cross-job interference often inflate these errors. As a result, robust systems must continuously audit estimator quality and fall back on conservative heuristics—such as adding safety margins—whenever error thresholds are exceeded. RL–based schedulers, including DeepRM, Decima, and various multi-agent variants, are well-suited for handling complex, multi-objective trade-offs. However, their effectiveness is heavily dependent on the fidelity of the training environment. Policies trained on synthetic or outdated traces often fail to generalize to live clusters, and achieving robust performance typically requires thousands of simulated episodes. This poses significant engineering overhead, especially when hardware or workload profiles evolve. Consequently, RL is most effective in stable environments (e.g., clusters with consistent mixes of DAG-structured DL

training jobs) and when high-quality simulators or extensive trace archives are available. Hybrid learning frameworks, such as Themis and Pollux, address these challenges by combining a lightweight ML tuner with a deterministic, rule-based core. In this setup, learning components adjust a small number of critical parameters—such as backfill thresholds or per-job resource ratios—while conventional logic ensures baseline correctness and guards against catastrophic model drift. Even in these hybrid systems, maintaining scheduler performance requires the tuner to effectively track changing workload dynamics. As such, modest but frequent online retraining—often daily or even hourly—may be necessary to sustain accuracy over time.

2.3. Evaluation Metrics and Validation Methods for Scheduling Models

Measuring the effectiveness of a scheduling model and any algorithms it inspires relies on well-defined metrics that capture both system efficiency and user-centric outcomes. Beyond the objectives listed in Table 3, scheduling for ML workloads introduces an important additional dimension: model quality. Optimizing solely for execution time can be misleading if, for example, resource allocations that minimize runtime degrade the final accuracy of a trained model. To address this, researchers have proposed ML-specific performance metrics. SLAQ assesses schedulers by the final training loss or accuracy achieved under a fixed resource budget, ensuring that faster completions do not come at the expense of model fidelity [108]. Pollux, on the other hand, defines *goodput* as the count of training samples that genuinely contribute to convergence, filtering out those wasted by suboptimal settings (e.g., excessively large batch sizes) [68]. These metrics are crucial whenever the ultimate goal of scheduling is not just speed but also the quality of the ML outcome.

Once suitable evaluation metrics have been defined, most studies adopt a three-pronged strategy for validating new scheduling models. First, *theoretical analysis* furnishes provable bounds on latency, fairness, or resource utilization under idealized assumptions, clarifying fundamental performance ceilings and exposing worst-case behaviors. Second, *simulation*—typically trace-driven or synthetic—offers a controlled sandbox in which competing designs can be stress-tested across diverse workload mixes and cluster sizes, enabling repeatable, apples-to-apples comparisons. Finally, *real-world deployment* on production clusters supplies the decisive reality check: it surfaces implementation overheads, robustness to unforeseen traffic spikes, and subtle interactions with existing infrastructure. Collectively, these layers of proof, in-silico experimentation, and live trials yield a comprehensive picture of each scheduler's strengths, limitations, and practical applicability.

Theoretical analysis: After selecting appropriate performance metrics, one method for evaluating a scheduling model or policy is through theoretical analysis. This approach entails proving bounds or optimality guarantees under the model's simplifying assumptions. For instance, one might show that a scheduling algorithm yields a makespan no greater than $1.5\times$ an optimal solution (i.e., a 1.5-approximation) on any input, or that under a given stochastic workload model, a policy minimizes expected response time. When models are sufficiently constrained—say, by assuming independent jobs or exact knowledge of runtimes—tools from classical scheduling theory or queueing theory can be applied to derive these results. Such proofs lend confidence by demonstrating, for example, that a new strategy enjoys strictly better worst-case performance or fairness properties than existing approaches. That said, these guarantees often depend on idealized conditions—e.g., i.i.d. job sizes or negligible context-switch overhead—that real GPU clusters may violate [129]. Consequently, theoretical insights are most compelling when paired with empirical validation.

Simulation: To assess scheduling models in settings closer to real-world deployments, researchers employ event-driven simulators that model a GPU cluster's behavior: jobs arrive, are queued, and then dispatched to virtual GPUs according to the scheduling policy, with completions and resource usage tracked precisely. Simulations may use synthetic workloads drawn from statistical distributions or replay traces of actual job arrivals and runtimes collected from production clusters [36,130]. Throughout each run, key metrics such as average job completion time, waiting-time distributions, GPU utilization over time, and fairness indices (e.g., Jain's index or the proportion of jobs meeting specified SLOs) are recorded [117]. By stress-testing algorithms under diverse conditions—such as workload surges or heavy-tailed job sizes—simulation reveals behavior that analytic methods cannot easily capture. Simulators vary in fidelity, from lightweight discrete-event frameworks that abstract away low-level details to comprehensive cluster emulators incorporating network topology and GPU memory contention [131]. Many studies feed in production traces to ensure their experiments mirror operational realities. Comparing strategies side by side in simulation highlights each approach's strengths and weaknesses (for example, excelling under high load but degrading when job sizes are unpredictable) and often drives further model refinement, such as adjusting assumed workload distributions to better fit observed data.

Real-world deployment: The most definitive validation of a scheduling model comes from deploying the scheduler on an actual GPU cluster and observing its performance under real workloads. Research prototypes are often tested on hardware ranging from small academic setups (tens of GPUs) to slices of production systems (hundreds of GPUs). For instance, the Tiresias scheduler was evaluated on a 60-GPU production testbed and demonstrated substantial reductions in median job completion time compared to the existing policy [74]. By running experiments on real clusters, one captures all the operational details and overheads that models and simulators may omit –including OS and driver overheads, resource-sharing effects, job startup latencies, and unpredictable user behavior. In these studies, new algorithms are typically benchmarked against standard baselines such as FIFO (first-in-first-out), dominant job first (DJF—a parallel-job variant of shortest-job-first), default Kubernetes or Slurm schedulers, and simple backfilling policies [36]. Researchers report not only average improvements but also gains at various percentiles (e.g., tail latency reductions) and analyze potential side effects such as whether optimizing for mean JCT disproportionately penalizes the longest jobs. These real-world experiments provide the strongest evidence that modeling and algorithmic innovations deliver tangible benefits in production environments.

In summary, validating GPU-scheduling models requires a three-pronged approach—proofs for ideal guarantees, simulations for controlled exploration, and real deployments for production realism—together providing the strongest evidence of practical utility.

2.4. Choosing a Scheduling Paradigm in Practice

To transform the catalog of paradigms in Section 2 into actionable guidance, we distill our empirical findings and surveyed case studies (Table 4) into a compact decision matrix. Practitioners can read the table top-to-bottom, stopping at the first row whose scenario description matches their workload. A short narrative justification follows Table 4.

Table 4. Decision matrix for selecting a GPU-cluster scheduling paradigm.

Workload Scenario	Primary Paradigm	Alternative Paradigm	Rationale
Single-tenant training-only, homogeneous cluster, preemptions costly	Greedy heuristic (e.g., SJF + backfill)	Queueing-index (SRPT/PS)	Low job-completion time (JCT) with minimal overhead; preemption avoidance simplifies implementation.
Multi-tenant mixed (training + inference), fairness critical, moderate load	DRF-based DP / optimization	Hybrid heuristic + ML-assisted prediction	Provides provable fairness guarantees (e.g., bounded max-slowdown); small throughput trade-off.
High load (>75% utilization), variable DAGs, preemption cheap	Reinforcement learning (e.g., Decima)	Hybrid heuristic (HEFT + backfill)	Learns long-horizon dispatch policies to reduce p95 latency; alternative achieves near-optimal makespan with lower engineering cost.
Real-time inference with strict SLOs (50∼200 ms), deadline-sensitive	Queueing-index (SRPT/processor-sharing)	Learning-assisted size prediction + PS	Size-based prioritization minimizes tail-latency; if exact sizes are unknown, predictions feed into PS to approach SRPT behavior.
Energy-budgeted training under power/carbon caps	MILP-based multi-objective optimization	Hybrid heuristic with DVFS	Precise modeling of time–energy trade-offs; alternative reduces solver latency while achieving near-optimal energy usage.
Offline batch scheduling with a fully known job set	Offline MILP or DAG-based optimization	Queueing-theory policies (e.g., $M/G/k$ with SRPT)	Computes optimal makespan for fixed jobs; when scale is large, queueing models offer analytical insights under stochastic approximations.
Unpredictable workloads, heterogeneous GPUs, noisy runtime estimates	Hybrid heuristic + ML prediction	Robust heuristic (backfill with conservative reservations)	ML-predictions guide placement, but the fallback heuristic prevents starvation when prediction error is high.
PDE / HPC solvers	Communication-aware MILP	Static bin-packing	Long-lived, tightly coupled jobs with predictable runtimes; performance dominated by NVLink/NVSwitch topology rather than queue dynamics.

Choosing a scheduling paradigm in practice depends on four interacting axes. First is the *load regime and preemption cost*: in lightly loaded or single-tenant clusters, classical greedy heuristics—such as Gavel's bipartite matching—achieve near-optimal makespan with minimal overhead. However, once utilization exceeds roughly 75%, reinforcement learning (RL) schedulers like Decima offer better adaptability to bursty arrivals and workload variability, consistently outperforming static rules though only after a warm-up period and with reduced interpretability. Second, the *objective hierarchy* determines whether fairness should outweigh throughput. In highly multi-tenant settings, DRF-aligned dynamic programming solvers can bound finish-time unfairness within a small constant factor of the leximin ideal. However, their exponential state space and reliance on exact runtime models confine them to small batches or partitioned sub-clusters. This limitation pushes larger deployments toward approximate or hybrid heuristics that trade a marginal degree of fairness for scalability. Third, *predictability and deadline sensitivity* shape inference

scheduling: token-level runtime predictors enable queueing-index policies that minimize p99 latency provided that preemption costs remain modest. These approaches are particularly effective for interactive or deadline-constrained serving. Finally, *global constraints*, such as power budgets or carbon caps, often favor white-box MILP formulations. Their linear structure allows precise modeling of time–energy trade-offs, whereas RL approaches typically converge more slowly on such global objectives. For workloads like PDE solvers or traditional HPC applications—characterized by long-lived, tightly coupled jobs with predictable runtimes—communication-aware MILP remains near-optimal. Similarly, stable inference farms benefit little from learning-based schedulers, given the regularity of their load profiles.

2.5. Comparative Analysis of Models

No single modeling approach is universally superior; each entails its own set of trade-offs. In what follows, we contrast the different models in terms of their information requirements, computational tractability, and practical applicability.

Offline models vs. online models: Offline models assume full knowledge of the workload in advance and can, at least in principle, compute schedules that are near-optimal for that fixed-job set. They shine in static or batch-processing contexts (for example, nightly jobs), where one can afford to spend significant computation time to derive an optimal plan. However, these approaches often struggle with both scalability and adaptability: a schedule that is optimal for a predetermined collection of jobs may perform poorly when faced with unexpected arrivals or delays.

By contrast, online models make decisions as jobs arrive, eschewing any prescience of future workloads [132]. Their primary goal is to sustain high utilization and low latency in dynamic environments. Lacking complete information, they typically employ heuristics or approximation algorithms; truly optimal online policies exist only under highly idealized assumptions. In practice, systems adopt robust "good-enough" rules that deliver reliable average performance. The strength of online models lies in their realism and flexibility—qualities essential for perpetually running services such as shared GPU clusters—although this comes at the expense of weaker theoretical guarantees.

In short, offline models are most valuable for controlled experiments or as benchmarking baselines, whereas online models are indispensable for real-time scheduling, trading some optimality for responsiveness and robustness in the face of uncertainty.

Queueing-theory-based models vs. optimization-based models: Queueing-theoretic approaches provide elegant analytical insights under stochastic assumptions (e.g., Poisson arrivals and exponential service times) [74,113]. In these simplified settings, they can identify optimal policies, such as SRPT or the Gittins index in an $M/G/1$ queue, and offer probabilistic performance guarantees (for example, bounds on average delay or queue-overflow probability). However, the abstractions that enable tractability often overlook key complexities: multi-GPU coordination, non-negligible setup times, and the heavy-tailed service distributions typical of ML training workloads.

By contrast, optimization-based models (e.g., MILPs or min-cost flow formulations) can encode detailed system characteristics without relying on simple stochastic assumptions. Taking deterministic inputs—whether point estimates or worst-case values—they capture GPU memory constraints, precedence relations in multi-stage jobs, fairness budgets, energy caps, and more. Their chief limitation is computational: they rarely scale to large, dynamic workloads without heuristic approximations or periodic re-solving, which may be impractical for real-time decision-making.

In essence, queueing models excel at providing probabilistic guarantees within idealized frameworks, while optimization models deliver high-fidelity scheduling at the expense

of scalability. Many production systems, therefore, combine both: leveraging queueing theory to establish high-level priority rules and employing optimization solvers to handle subproblems (such as packing jobs onto GPUs) at discrete intervals.

Heuristic approaches vs. formal methods: In practice, many GPU schedulers rely on fast, rule-based heuristics—such as greedy allocation to the best-fitting job or backfilling small jobs into idle slots—to make scheduling decisions with minimal overhead and implementation complexity [133]. These methods can be tuned via parameters (e.g., backfill aggressiveness or priority weightings) to match specific workload patterns or policy goals. Their main drawback is the absence of worst-case performance guarantees: under adversarial or highly skewed arrivals, a naive greedy policy may repeatedly starve large jobs or incur significant delays for certain job classes if fairness constraints are not carefully enforced.

By contrast, formal methods (e.g., integer linear programming or dynamic programming) can compute provably optimal schedules under a given model, ensuring that no better solution exists within the specified assumptions. However, their exponential time complexity generally precludes application to large or highly dynamic clusters except in offline calibration or small subproblem contexts. Furthermore, an "optimal" schedule derived from a simplified model may falter in practice if it omits factors such as communication overhead or variability in service times.

Consequently, state-of-the-art GPU schedulers often adopt a hybrid design: lightweight heuristics handle real-time, large-scale decision-making, while occasional formal optimizations tune parameters or solve constrained subproblems. This blended approach preserves the speed and robustness of heuristics, ensuring, for example, that GPUs are never oversubscribed and no job is indefinitely starved, while leveraging formal analyses to verify and improve the scheduler's performance in controlled scenarios.

Learning-based models vs. rule-based models: A growing trend is to infuse schedulers with machine learning, for instance, via reinforcement-learning–derived policies or predictive models that estimate job runtimes and performance across GPU types. Such learning-based schedulers can adapt automatically to workload dynamics and capture complex, nonlinear interactions (e.g., shifting bottlenecks between CPU, network, and GPU) that static heuristics often miss. In principle, as they accrue experience or improve their predictive accuracy, these models can converge toward near-optimal scheduling decisions.

However, learning-based approaches introduce significant overheads and operational challenges. Training an RL agent or regression model demands large historical datasets and substantial compute for simulation or model fitting, and there is frequently a *cold-start* phase during which performance may lag behind established baselines [134]. In production contexts, this initial under-performance can be unacceptable. Learned policies also tend to be opaque, making them difficult to interpret, debug, or formally verify factors that can undermine operator trust in high-stakes environments. Moreover, without careful regularization or retraining, models risk overfitting to past patterns and may fail to generalize when workloads or hardware configurations evolve (e.g., when new job types or GPU architectures are introduced).

By contrast, traditional rule-based schedulers—simple heuristics or parameterized policies—require no training data, are immediately operational, and degrade predictably only if their underlying assumptions are violated. They are straightforward to debug and tune by human operators and offer stable behavior even under unanticipated conditions.

Recognizing the strengths and weaknesses of both paradigms, recent work often adopts hybrid strategies: using ML to fine-tune heuristic parameters (such as backfill thresholds or time quanta) or delegating only specific subdecisions (e.g., selecting an optimal GPU type via a performance predictor) to learned components, while retaining

overall rule-based control. Such hybrids aim to harness the adaptability of learning-based methods without surrendering the reliability and interpretability of established heuristics.

Model selection: Scheduling models are most effective when aligned with the characteristics of their target environment. In batch-oriented HPC clusters—where hardware is homogeneous and throughput dominates—simple priority queues and backfilling heuristics remain the de facto standard. Cloud AI platforms, by contrast, must accommodate elastic scaling [135,136], heterogeneous GPU types, and multi-tenant fairness; here, optimization-based and learning-augmented approaches can yield substantial gains [137]. Real-time inference services impose strict SLOs and deadline guarantees, dictating the use of real-time systems theory—deadline-driven scheduling and preemptive policies—to ensure timely responses under all load conditions [84,138].

Each paradigm offers distinct trade-offs:

1. Queueing models – Pros: Analytically tractable; provide closed-form performance bounds and optimal policies under stochastic assumptions. Cons: Scale poorly to large systems without relaxation or decomposition; approximations may sacrifice true optimality.
2. Optimization models – Pros: Express rich constraints and system heterogeneity; deliver provably optimal schedules within the model's scope. Cons: Become intractable for large, dynamic workloads without heuristic shortcuts or periodic re-solving; deterministic inputs may not capture runtime variability.
3. Heuristic methods – Pros: Extremely fast and scalable; simple to implement and tune for specific workloads; robust in practice. Cons: Lack formal performance guarantees; require careful parameterization to avoid starvation or unfairness.
4. Learning-based approaches – Pros: Adapt to workload patterns and capture complex, nonlinear interactions (e.g., resource interference) that fixed rules miss. Cons: Incur training overhead and cold-start penalties; produce opaque policies that demand extensive validation to prevent drift and ensure reliability.

Because no single approach suffices for all scenarios, production GPU schedulers typically adopt a hybrid strategy: employing ILP or matching solvers for small-scale assignment tasks, relying on heuristics for real-time dispatch, and leveraging learning models for runtime and resource-usage prediction. This blend of methods—grounded in robustness, simplicity, and targeted optimization—enables high performance across diverse, large-scale workloads.

3. Scheduling Algorithms and Their Performance

With GPU-cluster scheduling formalized in Section 2, we now turn to the concrete algorithms that bring those models to life. This section surveys the primary algorithmic strategies for making scheduling decisions in GPU clusters. We organize our discussion into three broad families—classical optimization methods, queueing-theoretic techniques, and learning-based adaptive algorithms—each grounded in different modeling assumptions and offering distinct trade-offs. We then conclude by highlighting practical implementation challenges and presenting a comparative evaluation of their performance.

While models define objectives, constraints, and high-level frameworks, it is the algorithms that carry out the step-by-step assignment of jobs to GPUs. For example, an integer-linear-programming (ILP) formulation may be solved exactly with a MILP solver or approximately via greedy heuristics, whereas a Markov decision process (MDP) model can be tackled with a reinforcement-learning (RL) agent. Hybrid approaches—combining rule-based heuristics with machine-learning predictors—have also gained traction [139]. In the subsections that follow, we analyze each algorithm class in terms of design methodology,

computational complexity, and their ability to address the unique challenges of GPU cluster scheduling.

3.1. Classical Optimization Algorithms

Classical schedulers rely on deterministic rules or mathematical optimization, without any learning component. We begin with the simplest and still most widely deployed family.

Greedy algorithms: Greedy algorithms make locally optimal choices—such as scheduling the "best" job first according to a specific heuristic—to drive strong overall performance. A canonical example is shortest-job-first (SJF), which prioritizes jobs with the smallest runtimes. For a job set J with known processing times $\{p_j\}$ on a single GPU, SJF orders jobs by non-decreasing p_j. Denote by $C_j(\sigma)$ the completion time of job j under schedule σ. It is well known that SJF minimizes the sum of completion times, $\sum_{j \in J} C_j$, on a single GPU [140]. Despite its optimality for average completion time, SJF can starve long jobs [141]. Production systems typically counteract this by applying *aging* or enforcing a cap on maximum waiting time.

Backfilling: First-come-first-served (FCFS) with backfilling preserves arrival order for the head-of-queue job but allows smaller jobs to leapfrog when resources are idle [119]. If the front job h is blocked by insufficient GPUs, the scheduler reserves resources for h at its earliest start time, then scans the queue for a shorter job j that can complete before h's reservation. This eliminates idle gaps while ensuring h is never delayed.

Packing on multiple GPUs: When more than one identical GPU ($m > 1$) is available, placement reduces to a bin-packing problem that is typically handled with simple list-scheduling heuristics. The first-fit (FF) strategy scans the job queue in order and assigns the earliest job whose demand g_j can be satisfied by the current set of free GPUs B, whereas best-fit (BF) looks only at the jobs that fit and chooses the one that leaves the smallest residual capacity $|B| - g_j$. Implemented with a heap, both heuristics run in $O(|J| \log |J|)$ time, and Graham's classic bound guarantees that each achieves a $\left(2 - \frac{1}{m}\right)$-approximation of the makespan for identical machines, with BF inheriting the same ratio by dominating FF [142].

Heterogeneous clusters: On clusters with diverse GPU types $T = \{\tau_1, \ldots, \tau_s\}$, naive FF/BF degrade because a job's throughput varies by device. Gavel [57] addresses this by probing each job's per-device throughput $\phi_{j,\tau}$ and scheduling in the transformed space of *effective GPU-seconds*, thereby extending classical packing heuristics while preserving fairness.

Scalability: All above heuristics maintain a priority queue or balanced tree, incurring $O(\log |J|)$ overhead per scheduling event. This yields sub-millisecond decision latencies even with hundreds of queued jobs—critical for environments with frequent arrivals and completions. Their main limitation is the lack of worst-case guarantees beyond Graham's bound and potential starvation under pathological workloads. Nevertheless, empirical studies show that well-tuned greedy rules strike a favorable balance between utilization and responsiveness in production clusters (for deep learning supercomputers, MARBLE refines FF by dynamically resizing each job's GPU allocation using its empirical scalability curve, further boosting throughput [143]).

Recent advances also target multi-GPU deep-learning workloads in HPC settings. For example, MARBLE [143] dynamically determines the optimal number of GPUs per job—based on pre-profiled scalability curves—and employs suspend/resume operations to co-schedule multiple jobs on the same node, exploiting non-linear scaling to reduce overall completion times on systems like Summit.

Dynamic programming: When local heuristics are insufficient, researchers have explored exact but more computationally expensive methods, such as dynamic programming

(DP). DP computes exact schedules for a variety of small-scale subproblems, at the cost of exponential or high-polynomial time. Consider a job set J with release times r_j, deadlines d_j, and processing times p_j on a single GPU. Define $U_j = 1$ if the job j is finished by its deadline and $U_j = 0$ otherwise. The feasibility problem $1|r_j, d_j|\sum_j U_j$—selecting the largest subset of jobs that meet their deadlines—admits an $O(2^{|J|}, |J|)$ state-space DP, while its non-preemptive and preemptive generalizations can be solved in $O(n^7)$ and $O(n^{10})$ time respectively [144]. When deadlines and processing times are small integers, a time-indexed DP yields a pseudo-polynomial algorithm, making it practical for moderate-scale instances.

Although full DP is often infeasible at large scale, it is optimal for key cases. For example, earliest-deadline-first (EDF) is provably optimal for preemptive scheduling with deadlines on a single machine [145]. In the non-preemptive setting, DP can enumerate subsets in increasing order of d_j to maximize throughput exactly, but its exponential term confines its use to small job sets. Beyond ordering, DP also applies to resource-allocation subproblems: it can jointly optimize GPU-count assignments, batch sizes, or parallelization granularity for individual jobs by exploring all allocation states [137].

In production GPU schedulers, DP is often invoked selectively within hybrid frameworks. For instance, OASiS embeds a polynomial-time DP inside an online primal–dual algorithm to determine worker/parameter-server allocations that maximize aggregate utility under time-varying capacities [146]. The DP runs only for a small set of high-priority jobs, while lower-priority jobs revert to greedy heuristics. This "bounded DP + greedy" approach achieves near-optimal throughput with per-decision latencies below 100ms on real cluster traces.

ILP/MILP formulations and solvers: MILP provides a general framework for scheduling: introduce a binary decision variable $x_{j,k,t}$ for each job j, GPU k, and start time t; add linear constraints for assignment, capacity, and makespan; and use a MILP solver to search for the optimal solution. For identical GPUs and makespan minimization $P||C_{\max}$, define

$$J = \{1, \ldots, n\}, \quad G = \{1, \ldots, m\}, \quad T = \{0, \Delta, 2\Delta, \ldots, H\},$$

and

$$x_{j,k,t} = \begin{cases} 1 & \text{if job } j \text{ starts on GPU } k \text{ at time } t \\ 0 & \text{otherwise} \end{cases}$$

Then the time-indexed MILP is

$$\begin{aligned}
&\min C_{\max} \\
&\text{subject to} \sum_{k \in G} \sum_{t \in T} x_{j,k,t} = 1 && \forall j \in J && \text{(assign)} \\
&\sum_{j \in J} \sum_{\tau = \max\{0, t - p_j + \Delta\}}^{t} x_{j,k,\tau} \leq 1 && \forall k \in G, t \in T && \text{(capacity)} \\
&t + p_j x_{j,k,t} \leq C_{\max} && \forall j \in J, k \in G, t \in T && \text{(makespan)} \\
&x_{j,k,t} \in \{0, 1\} \\
&C_{\max} \geq 0
\end{aligned}$$

This formulation uses $O(n, m, |T|)$ binary variable and is strongly NP-hard even for $m = 2$ [147]. Generic branch-and-bound solvers (Gurobi, CPLEX) typically handle up to ~ 50 jobs and ~ 10 GPUs before runtimes become prohibitive. Two common relax-and-round strategies mitigate this:

1. Continuous relaxation: Replace $x_{j,k,t} \in \{0, 1\}$ with $0 \leq x_{j,k,t} \leq 1$, solve the LP, then round the fractional solution.

2. List-scheduling rounding: Sort jobs by LP-derived completion times $\hat{C}_j$ and assign greedily to the earliest available GPU.

When release times are present, online list-scheduling variants achieve a 2-competitive ratio [148,149]. Although full MILP is rarely used online, it serves as a ground-truth oracle for benchmarking heuristics under simplified assumptions [57].

Structure-exploiting solvers: Special cases admit faster combinatorial algorithms. If preemption is allowed and communication costs are ignored, each scheduling epoch reduces to a bipartite matching problem, solvable via the Hungarian algorithm in $O(n^3)$ time [124]. For single-node multi-GPU placement with memory constraints, column-generation generates only profitable assignment patterns, closing much of the optimality gap while keeping per-decision latency under one second for up to 200 jobs [150].

In summary, ILP/MILP provides a rigorous benchmark and a foundation for principled approximation schemes (LP-rounding, list scheduling), though direct deployment at cluster scale is limited by computational cost.

Heuristic and meta-heuristic approaches: Beyond simple greedy rules, meta-heuristics such as genetic algorithms (GA), simulated annealing (SA), Tabu search, and particle-swarm optimization (PSO) have been explored for GPU scheduling, especially in heterogeneous environments. These methods encode a complete schedule (or resource allocation) as a candidate solution and iteratively refine it.

In a GA, for instance, each chromosome may represent an assignment of jobs to time slots or GPUs. An initial population of random schedules undergoes crossover and mutation, guided by a fitness function aligned with objectives like minimizing makespan or maximizing throughput. GAs have been applied to CPU–GPU cluster scheduling to optimize multiple criteria simultaneously—such as makespan, energy efficiency, and fairness [107].

SA, by contrast, perturbs a current schedule with local random moves and accepts changes according to a temperature-driven probability: uphill moves are occasionally allowed to escape local optima, with the acceptance probability decreasing over time. Similar strategies underlie Tabu search and PSO.

Meta-heuristics excel at exploring large, complex search spaces and can produce high-quality schedules given sufficient runtime. However, like MILP solvers, they incur significant computational overhead—often seconds to minutes per schedule—making them unsuitable for online scheduling, where decisions must occur within milliseconds. As a result, their use is generally confined to offline or semi-offline settings (e.g., precomputing schedules for known workloads or tuning parameters for lighter-weight heuristics). Furthermore, they lack optimality guarantees, and their performance hinges on careful parameter tuning, which itself can be costly [151]. Empirical studies show that GA and SA may slightly outperform greedy heuristics in heterogeneous clusters, but at the expense of minutes of computation time per run.

Despite these drawbacks, meta-heuristics offer valuable insights for policy design. For example, one could run a GA offline on historical traces to optimize scheduling-policy parameters, then deploy the tuned heuristic for real-time decisions. In practice, many production-grade managers—such as Slurm and Kubernetes—rely on simple heuristics augmented with tunable weights or priorities, striking a balance between responsiveness and solution quality [152].

3.2. Queueing-Theoretic Approaches

Queueing-theoretic scheduling algorithms draw on analytical results from classical queueing models to assign each job a dynamic priority. Instead of solving a global optimization, these policies maintain a simple "index" or rank for every job—based on its

service history or statistical characteristics—and always dispatch the job with the highest priority. Below, we outline four influential families of queue-based strategies.

Index policies (e.g., Gittins index): Index policies compute a per-job score that balances fairness and efficiency by considering both attained service and the likelihood of short remaining work. The Gittins index is a canonical example: for an $M/G/1$ queue, a job with attained service v receives a Gittins index

$$\mathcal{G}(v) = \sup_{\tau>0} \frac{\Pr\{\, S_j \le v + \tau \mid S_j > v\}}{\mathbb{E}[\, \min(\tau,\, S_j - v) \mid S_j > v]},$$

where S_j is the (unknown) total service time of job j and $\mathbb{E}[\cdot]$ denotes expectation. Here, the numerator is the probability that the job will complete within the next τ units of service given that it has already consumed v; the denominator is the expected additional service it would require over that same horizon. The job with the highest index is scheduled next, yielding a policy that is provably optimal for minimizing mean slowdown when job sizes are unknown, even though exact computation is often approximated in practice [153].

Least attained service (LAS) / foreground–background (FB): A simpler offshoot of the Gittins family sets $\mathcal{G}(v) = -v$, yielding the LAS/FB rule: always run the job with the smallest accumulated service. Under heavy-tailed service distributions, LAS incurs at most a 25% penalty over SRPT for mean slowdown—the information-theoretic lower bound [154]. New arrivals preempt longer-served jobs immediately, mimicking multi-level feedback. In GPU clusters, full preemption is often too costly, so systems like Tiresias use fixed-length leases: after each time slice, the job with least total GPU-time may preempt the current one. By tracking both time and GPU count (2D-LAS), Tiresias avoids starvation with minimal overhead. E-LAS further extends this by incorporating real-time epoch progress rates into the priority, improving job-completion times without runtime profiling [155].

Multi-class priority queues: Queueing theory also motivates dividing work into several priority classes, e.g., "urgent," "standard," and "background." Strict priority in an $M/M/1$ system guarantees minimal response for the top class, though lower classes suffer increased delays. GPU schedulers often emulate SRPT by maintaining, say, short-job and long-job queues: serve all jobs with estimated runtime $< X$ seconds first, then switch to longer tasks. Thresholds are set based on historical job-size distributions, and user-provided estimates can refine class assignments [156].

Processor sharing (PS) approximations: Ideal processor sharing divides capacity equally among active jobs, preventing starvation and smoothing progress. GPUs cannot preempt at the granularity PS demands, but features like NVIDIA A100's MIG partitions approximate independent instances. Alternatively, coarse time slicing or round-robin dispatchers (e.g., Themis's finish-time equalization) reassign GPUs periodically to equalize job progress—capturing PS's fairness spirit while limiting I/O and checkpointing overhead [85]. It is worth noting that these policies are derived under single-server assumptions. In multi-GPU clusters, naively applying them per GPU may break global optimality. Practical extensions either decentralize the policy (one index per GPU) or coordinate across servers to generalize index and priority rules to the multi-machine setting.

Practical applicability of queueing models: Queueing-based algorithms come with strong theoretical guarantees under idealized models, but real GPU clusters demand careful approximations and tuning. For example, LAS requires choosing a lease duration: if leases are too long, short jobs lose their responsive service; if too short, preemption overhead grows. A concrete instantiation—"preempt every 30s to rebalance toward least-attained service"—captures LAS behavior in practice and can be tuned analytically or via empirical evaluation [85]. Likewise, Gittins-style policies rely on accurate size distributions; misestimating a job as short may cause it to monopolize GPUs. Tiresias experiments show

that both partially informed (Gittins-like) and agnostic (LAS-style) schemes outperform classical heuristics, suggesting that even simplified index rules can realize much of the theoretical benefit in realistic workloads.

Predictive admission and QoS control: Beyond scheduling, queueing theory's formulas can drive admission control and service-level guarantees. A scheduler that estimates, say, a one-hour wait for newly submitted work can use that prediction to deny, defer, or reprioritize jobs based on user deadlines or QoS contracts. Simple metrics—Little's Law ($L = \lambda W$), mean response-time bounds, or $M/M/1$ approximations—can trigger autoscaling or more aggressive preemption when saturation looms, helping to balance cluster utilization against developer expectations.

Trade-offs vs. optimization-based methods: Queueing policies like LAS and Gittins Index optimize user-centric metrics (mean response time, slowdown) at the expense of additional preemption and decision overhead. In contrast, optimization-based schedulers typically focus on system-centric objectives (throughput, GPU utilization), often allowing long jobs to finish uninterrupted to avoid migration or checkpoint costs. For instance, an MILP solver may keep a nearly completed job running, boosting overall throughput but delaying short arrivals—precisely the opposite of SRPT-like fairness.

Hybrid designs and practical deployment: Most production schedulers blend both worlds: they pack GPUs efficiently and layer in preemption or time-slicing to prevent starvation. Gandiva, for example, uses placement-optimization via migration while time-slicing hyperparameter searches for rapid feedback [56]. A common pattern is to treat each allocated GPU as a "service chunk" when computing priorities, enabling index or LAS heuristics to work atop any resource-packing framework.

In summary, queueing-theoretic approaches bring a rich repertoire of priority rules, fairness guarantees, and predictive insights. Their core strength lies in optimizing responsiveness and equity—qualities highly valued by ML practitioners—while their main challenge is controlling overhead and ensuring system-wide efficiency. As demonstrated by Tiresias and Themis, with careful adaptation and cluster-specific enhancements, queueing-inspired schedulers can substantially reduce average completion times and deliver fair, predictable GPU allocation in multi-tenant environments [74,85].

3.3. Learning-Based Adaptive Algorithms

Learning-based algorithms harness machine learning—via supervised prediction or reinforcement learning—and adapt over time using feedback from historical logs or live system metrics [157–159]. Rather than relying on fixed rules, these methods continually refine their policies as workload patterns evolve.

Supervised prediction models: Predictive models are trained on job features—such as neural network architecture, input size, and user identity—to forecast metrics like runtime, memory footprint, or scaling behavior. A scheduler can then implement a data-driven SJF by ordering jobs according to predicted runtimes. To mitigate mispredictions, jobs that exceed their expected duration are demoted to a lower-priority queue, preserving fairness and robustness.

In heterogeneous GPU clusters, models estimate performance across hardware types. Gavel benchmarks workloads to derive throughput estimates for A100 versus V100 GPUs, adapting classical policies to maximize efficiency in mixed environments [57]. Likewise, GENIE uses lightweight profiling to predict deep learning job latencies and throughputs, enabling a QoS-aware scheduler that meets latency and throughput targets [160].

Interference prediction further enhances sharing. Co-scheML profiles ML applications offline to learn co-location slowdown patterns and predicts whether a new job can safely share a GPU without degrading performance [161]. Remaining-time predictors leverage

in-flight metrics—epochs completed or loss curves—to decide if a job should be allowed to finish or be preempted, as in SLAQ's loss-gradient prioritization [108]. Forecasting tools like Bamboo and Parcae anticipate spot-instance revocations, adjusting parallelism ahead of interruptions to maintain throughput [162].

By replacing conservative or user-provided estimates with learned predictions, supervised methods often outperform traditional backfilling—moderate accuracy alone can yield significant reductions in waiting time and improved resource utilization.

Reinforcement learning–based scheduling: RL casts scheduling as a sequential decision problem: the scheduler (agent) observes the cluster state (job queue, GPU availability), takes actions (dispatch or preempt jobs), and receives rewards reflecting metrics like negative waiting time, throughput, or fairness. Over repeated interactions, the agent refines its policy to maximize cumulative reward.

Early systems such as DeepRM [82] tame the combinatorial state by limiting the number of waiting jobs and discretizing time and resource slots. They encode the cluster as an "image" of resource occupancy plus job demands and train a neural network to output a scheduling distribution. Decima extends this idea to DAG-structured workloads: a graph neural network embeds each job's task graph and a policy network selects which task or workflow to advance next, outperforming heuristics like SRTF and fair sharing under variable load [127]. Likewise, HeteroG applies a GNN-based RL agent to jointly learn device placement, parallelism, and communication strategies for DNN (deep neural network) training DAGs [163]. RL's flexibility also enables multi-objective rewards—for example, minimizing average slowdown minus a fairness penalty—which can reveal nuanced trade-offs beyond fixed heuristics. Ryu et al. [164] demonstrate this by training an LLM scheduler to avoid co-scheduling network-heavy jobs on the same switch, reducing contention and boosting throughput.

Despite its promise, deploying RL on live clusters faces four intertwined challenges. Exploration-versus-exploitation tension forces policies to try suboptimal actions to improve over time, but this experimentation can disrupt tenants. As a result, most practitioners pre-train offline—using simulators or historical traces—and then periodically fine-tune as hardware or workloads evolve. State-space explosion occurs because real clusters manage hundreds of concurrent jobs. To address this, compact abstractions like DeepRM's image grids, Decima's graph embeddings, and transformer-based attention mechanisms help focus the agent on key features and keep learning manageable [165]. Stability and safety require that learned strategies adhere to hard constraints—such as maximum wait times or per-tenant GPU caps—so modern systems often embed the RL policy within a rule-based guardrail that ensures correctness before actions are executed. Finally, interpretability remains a challenge: opaque policies make it difficult to understand why a specific scheduling decision was made, undermining operator trust in mission-critical environments [166]. Despite these hurdles, momentum is building. Industrial teams at Alibaba and several hyperscalers are already prototyping deep RL for resource management, motivated by the steady improvements observed in research prototypes.

Adaptive scheduling via feedback control: Not all adaptive scheduling approaches rely on machine learning; some use principles from feedback control systems [167–169]. These methods continuously monitor system metrics and adjust scheduling behavior in response, forming a closed-loop system. For instance, a basic feedback-driven scheduler might monitor average GPU utilization over the last minute: if it falls below a predefined threshold, the system could respond by increasing GPU packing density (e.g., enabling more aggressive time-sharing) or by admitting more jobs from the queue. Conversely, if the average job slowdown rises above a target, the scheduler could throttle job admissions or boost the priority of small or short jobs. Such strategies can be viewed as control systems:

monitor a key performance indicator, and if it deviates from the target, adjust a parameter in the scheduling policy accordingly.

Similarly, Elan [169] provides efficient elastic scaling for deep learning jobs by combining a hybrid batch size scaling mechanism with asynchronous coordination and topology-aware state replication. Unlike checkpoint-based systems, Elan allows jobs to scale out, scale in, and migrate without restarting, minimizing disruption and achieving sub-second elasticity with negligible overhead. By balancing training efficiency and model convergence during resource changes, Elan enables higher GPU utilization and faster job completion in production-scale clusters.

A more sophisticated example of feedback control in GPU scheduling is incremental job scaling, as implemented by systems like Pollux and AntMan [68,168]. Pollux uses an online feedback loop per job to monitor its goodput (effective throughput in terms of training progress per unit time) and adapt GPU allocations accordingly. If additional GPUs do not significantly improve goodput (i.e., the job hits diminishing returns), the system reduces its GPU share and reallocates resources to other jobs. Conversely, if a job scales efficiently, more GPUs are allocated up to the point of diminishing returns. This feedback-driven adjustment occurs periodically and adapts to the evolving performance characteristics of each job. AntMan adopts a similar philosophy but extends it by modifying the underlying deep learning frameworks to allow fine-grained, real-time scaling of both GPU memory and compute resources. By continuously monitoring mini-batch performance and dynamically adjusting resource allocation without requiring job restarts, AntMan opportunistically improves GPU utilization and cluster throughput under multi-tenant workloads. Both systems exemplify adaptive behavior based on online measurements—a form of reinforcement through iterative performance feedback—positioning them between classical control and learning-based approaches, and leveraging empirical tuning without requiring offline model training.

Another direction is predictive queueing control, where the scheduler uses observations of job inter-arrival and service times to anticipate load patterns and adjust parameters preemptively. For example, if high-priority jobs begin arriving more frequently, the scheduler might increase the time-slice frequency to enhance multitasking capability or temporarily preempt low-priority jobs to make room. These techniques combine elements of prediction and feedback, aiming to dynamically stabilize key system metrics (e.g., response time, fairness) under workload fluctuations.

Scalability and practical deployment of learning-based scheduling: Once trained, learning-based schedulers can make decisions in milliseconds—often accelerated further on GPUs—easily keeping pace with the few dozen scheduling events per second typical in production clusters. The principal overhead lies in training these models and in collecting representative data that captures the full variability of the target environment. A major scaling challenge is handling a dynamic and potentially very large job set: some designs address this by truncating inputs to a fixed-size "window" of the most urgent or largest jobs and summarizing the remainder, while others employ flexible architectures—such as transformers that natively accept variable-length sequences [170]—or iterative decision loops that process one job at a time. In clusters on the order of 10,000 GPUs, even these techniques can strain centralization; *multi-agent reinforcement learning* (MARL), in which each agent oversees a rack or node group and coordinates via shared rewards or a meta-policy, offers one promising path toward decentralized, scalable control.

Several RL-based prototypes have validated this approach in simulation. DeepRM and Decima showed that learned policies can surpass classical heuristics under complex constraints like DAG dependencies and heterogeneous job mixes [82,127]. More recently, HeterPS employed an LSTM-based RL scheduler to allocate tasks across CPUs and GPUs

in heterogeneous clusters, achieving faster end-to-end training times [171]. These studies demonstrate that, when properly trained, RL can internalize intricate scheduling dynamics and adapt to shifting workloads more effectively than static rules.

Despite these advances, industrial uptake remains cautious. Scheduling is mission-critical, and missteps risk severe performance degradation or outages. Consequently, most production systems employ ML components in a hybrid fashion—using learned models for parameter tuning or runtime prediction, but retaining proven control logic as the primary decision maker—and incorporate extensive safety checks, fallback paths, and human-in-the-loop oversight before permitting fully autonomous operation.

3.4. Comparative Analysis

In this subsection, we evaluate the major categories of scheduling algorithms according to four key dimensions: computational complexity, performance characteristics, scalability, and the environments in which they excel.

Complexity and decision speed: Classical heuristics—such as greedy or rule-based methods—offer extremely low decision latency, often operating in $O(\log n)$ time when using priority queues. Consequently, they can react to job arrivals and completions almost instantaneously relative to typical runtime durations. By contrast, ILP- and MILP-based approaches incur substantially greater computational cost: solving a moderately sized MILP can require seconds or even minutes, rendering these methods impractical for real-time scheduling events. Nevertheless, they remain viable for coarse-grained optimization tasks—for example, computing an optimal allocation hourly. Metaheuristics like GA and SA similarly demand considerable time to converge, making them better suited for offline analysis or periodic batch scheduling rather than real-time decision-making. Queueing-theoretic policies (e.g., LAS or Gittins index) perform lightweight per-job updates—such as tracking cumulative service—at negligible expense. Likewise, RL policies, once trained, execute scheduling decisions with minimal overhead: neural network inference or table lookups typically complete in milliseconds or less on contemporary hardware. Overall, heuristics, queueing strategies, and trained RL models scale efficiently to thousands of jobs with low runtime overhead, whereas ILP formulations and iterative optimization techniques struggle to scale beyond a few hundred jobs without substantial simplification. For instance, DynamoML integrates fractional GPU sharing, workload-aware gang scheduling, and SLA-driven auto-scaling as Kubernetes operators, yet still achieves per-event decision times under 100ms while managing hundreds of concurrent jobs in real clusters [172].

Quality of schedule: When ranked by theoretical proximity to optimal performance, exact ILP solutions remain unsurpassed within their modeling assumptions, although they often omit system-level factors such as preemption costs and workload heterogeneity.

RL schedulers can empirically approach optimality when supplied with abundant training data and expressive policies, yet formal convergence guarantees are rare and highly sensitive to the training regime. Lightweight greedy heuristics deliver strong average-case results but can deviate markedly from optimality on adversarial inputs, while queueing-theoretic rules—most notably the Gittins index—offer provable optimality for specific objectives (e.g., minimizing mean slowdown in a single-server queue) but may sacrifice utilization or strict fairness in broader settings. Empirical evidence thus reveals inherent trade-offs: policies that aggressively pack GPUs to maximize utilization often prolong waits for small jobs, whereas LAS-like time sharing improves responsiveness at the cost of resource efficiency. Hybrid methods attempt to reconcile these goals, mitigating under-utilization while preserving fairness and interactivity.

Choosing a concrete algorithm therefore hinges on the dominant performance goal. To minimize average job-completion time (JCT), size-based disciplines such as SRPT or Gittins-

index scheduling are theoretically optimal when job sizes (or their distributions) are known, and in practice shortest-job-first (SJF) supplemented with accurate runtime predictions achieves comparable gains [74]. To maximize utilization, dense-packing strategies—best-fit placement, bin-packing heuristics, and non-preemptive FCFS or backfilling—maintain high GPU occupancy and limit idle gaps. To ensure fairness, time-sharing mechanisms such as LAS or round-robin prevent starvation, while cluster-level systems like Themis employ auction-based allocation to provide explicit fairness guarantees at the cost of additional coordination overhead [85]. Finally, when multiple objectives must be balanced, heuristic frameworks blend metrics through weighted composite priorities, and RL policies achieve similar trade-offs via reward shaping or exploration strategies such as the "power of two choices," dynamically discovering schedules that balance responsiveness, utilization, and equity.

Scalability and heterogeneity: Lightweight scheduling algorithms naturally scale to large clusters, since event rates—job arrivals, completions, and preemptions—are typically limited by job runtimes. Even in a busy 1000-GPU deployment, only tens of scheduling events occur per second, allowing a single-threaded priority–queue or placement update to keep pace. At web-scale (e.g., Google's Borg, managing millions of jobs), however, centralized schedulers become a throughput bottleneck, prompting distributed or hierarchical designs that partition the cluster into independently managed cells. Most academic proposals assume one scheduler handling a few hundred to a few thousand machines, a regime where heuristics and RL policies remain practical; by contrast, ILP and metaheuristic methods incur prohibitive computation at this scale.

RL approaches can scale in principle, but only if the policy efficiently summarizes system state. Naively encoding all jobs and resources leads to state-space explosion. To mitigate this, many RL schedulers use fixed-size job subsets, recurrent or attention-based networks, or other compact representations. Some systems train on small clusters and transfer policies to larger ones, though such generalization often demands careful architecture design and retraining.

Heterogeneity in GPU types, network bandwidths, and workload characteristics further complicates scaling. Heuristics and queueing frameworks adapt readily—by maintaining per-resource queues or weighting priorities by expected slowdown on each GPU class. ILP formulations can model heterogeneity via extra constraints, but at the cost of larger variable counts and slower solves. RL methods can encode resource types directly in the state, yet typically require retraining to master a heterogeneous environment. Notwithstanding this engineering overhead, recent work such as HeterPS demonstrates that RL can effectively learn in diverse cluster settings [171]. Overall, simple heuristics offer the greatest flexibility for rapid deployment, while RL and optimization-based techniques, once tuned or trained, can yield superior performance in large, heterogeneous clusters.

Other performance metrics: fault tolerance, interpretability, explainability, and transparency: Simpler scheduling algorithms are inherently easier to make fault-tolerant. For example, after a crash or restart, a greedy or rule-based scheduler can rebuild its internal state—essentially a queue of pending jobs—by querying the current cluster status. RL agents, by contrast, do not carry over internal state between decisions (beyond what the environment encodes); their policies must simply be persisted to avoid retraining, which is generally straightforward since they remain static during deployment.

Optimization-based methods (ILP/MILP) present greater recovery challenges. Any change in system state may force the solver to restart from scratch to compute a new solution, a process that can be time-consuming. In the event of infrastructure failures—such as a GPU or node outage—most schedulers treat these incidents as preemptions: the affected job is re-queued or scheduled elsewhere. Heuristic approaches often include built-

in fallback rules (e.g., deprioritizing unreliable nodes), whereas learning-based methods can fold failure events into their training or apply online policy updates to adapt over time.

Interpretability and transparency are also critical in multi-tenant clusters, where users' trust depends on understanding scheduling decisions. Rule-based and heuristic policies are simple to explain—"short jobs first," "equal GPU shares per user"—enabling clear mental models and reducing frustration. In contrast, ML-driven schedulers, especially RL policies, can exhibit behavior that is hard to trace back to explicit rules. Even when optimizing a well-defined multi-objective function, their decisions may seem arbitrary, which can undermine user confidence when one job is delayed while another proceeds.

3.5. Cross-Paradigm Empirical Comparison

The previous discussion compared scheduling paradigms primarily on qualitative grounds such as scalability to thousands of GPUs, fault tolerance, and the transparency of scheduling decisions. To complement that narrative, we now turn to a quantitative perspective: given a representative workload, which paradigms deliver the greatest gains in latency, throughput, and fairness? Table 5 presents key performance metrics from representative algorithms within each paradigm. Metrics are normalized against baseline approaches (e.g., heuristic or default schedulers) to emphasize relative improvements. Latency measures job completion time (lower is better), throughput/utilization captures jobs completed per unit time or resource efficiency (higher is better), and fairness reflects how well each approach maintained equitable resource sharing.

Table 5. Cross-paradigm empirical comparison of GPU scheduling algorithms. Improvements are reported relative to each study's baseline (↑ means higher/better, ↓ means lower/better).

Paradigm	Representative Algorithms/Workloads	Latency Improvement	Throughput/Utilization	Fairness Impact
Greedy heuristics	Tiresias [74] (LAS, training); Gandivafair [70] (time-sharing)	≈5.5× ↓ mean JCT vs. fair baseline; long-job slowdowns mitigated	High GPU utilization via backfilling; heterogeneous GPUs reused via token trading	Maintains long-term fair share
Dynamic programming	Lyra [173] (capacity loaning, mixed training + inference)	(1.48∼1.53)× ↓ queuing/JCT	Up to 25% ↑ GPU utilization when borrowing idle inference GPUs	Inference SLOs preserved
MILP optimization	Chronus [84] (deadline, training); Dorm/AlloX (fair allocation)	14.7× ↓ deadline misses; 19.9× ↓ best-effort JCT	Near-optimal utilization; MILP solved in batches (solver overhead acceptable)	Strong DRF-level fairness
Queueing theory	LAS priority (Tiresias, training) [74]; dynamic batching model (inference)	(5∼6)× ↓ short-job wait; latency vs. batch size analytically predictable	Up to 13× ↑ throughput at large batch; short-job throughput boosted	Possible long-job slowdowns if not combined with fairness controls
ML-assisted prediction	Helios [106] (runtime/priority); SCHEDTUNE [174] (interference, memory usage)	≈44% ↓ average JCT; 17.5% ↓ makespan	81% ↑ GPU-mem utilization; avoids OOM; better packing	Generally fair with tuned safeguards
Reinforcement learning	DL2 [159] (supervised, + RL, training); SCHED^2 [157] (Q-learning)	44.1% ↓ average JCT vs. DRF; 17.5% ↓ vs. expert heuristic	1.8× ↑ jobs / hour; lower fragmentation	Fairness achieved when encoded in reward
Hybrid approaches	Pollux [68] (adaptive goodput); AntMan [168] (elastic scaling)	(1.2∼1.3)× ↓ average JCT	1.5× ↑ throughput; (25 ∼ 40)% ↑ goodput	Balances efficiency & fairness via policy knobs

We summarize the absolute performance of representative GPU scheduling systems in Table 6.

Table 6. Absolute performance summary of representative GPU scheduling systems. Metrics are reported exactly as published; queueing delay and job completion time (JCT) are in seconds unless otherwise noted (↑ means higher/better, ↓ means lower/better).

System	Workload/Testbed	Representative Absolute Metrics
Tiresias-G, Tiresias-L	64-GPU ImageNet-style training trace	Average queueing delay: 1005 s (G), 963 s (L); median 39 s/13 s. Small-job JCT: 330 s (G), 300 s (L). Workload makespan: 27,510∼27,400 s vs. 33,270 s (YARN-CS baseline).
Llumnix	16× LLaMA-7B inference cluster	Live-migration downtime: 20∼30 ms vs. 3.5 s recompute. P99 first-token latency: up to 15× lower than INFaaS. Up to 36% fewer instances at equal P99 latency.
Pollux ($p = -1$)	64-GPU synthetic training workload	Average JCT: 0.76 h; P99 JCT: 11 h; makespan: 16 h.
DL2	64-GPU parameter-server testbed	Scheduler latency: 0.7 s. GPU utilization: 78% (↑ 16% vs. DRF). Scaling overhead: 0.4% of total training time.
Lyra	15-day simulation (3544 GPU training + 4160 GPU inference)	Average queueing time: 2010 s vs. 3072 s (FIFO). Average JCT: 11,236 s vs. 16,610 s. Cluster utilization: 86% vs. 72%.
Chronus	120-GPU Kubernetes prototype	Deadline-miss rate: 5% (≈14.7× ↓ vs. prior). Pending time cut from 2105 s to 960 s. Best-effort JCT: up to 19.9× faster.
Helios	1 GPU + 12 SSD GNN training	Throughput: up to 6.4× GIDS, 182× Ginex. Saturates PCIe version 4.0 × 16 with 6 SSDs; 91∼99% of in-memory throughput on PA dataset.

Design takeaways. Greedy heuristics offer robust, dependable performance across a wide range of applications and are relatively easy to implement. However, they typically require manual tuning for each deployment context, and their fairness can degrade under bursty or heterogeneous workloads. In contrast, MILP approaches can achieve the lowest latencies at small to moderate cluster scales, but their computational cost increases sharply with scale, making them impractical for large deployments. Finally, RL policies often face a cold-start challenge, performing poorly until sufficient experience is accumulated. Once trained, however, they can uncover and exploit complex patterns among jobs, resources, and placement decisions—ultimately matching or even surpassing other methods in steady-state performance.

3.6. PDE-Driven HPC Workloads

Many scientific applications—such as compressible-flow hydrodynamics, finite-difference time-domain (FDTD) electrodynamics, magnetohydrodynamic plasma, and seismic wave propagation—solve systems of coupled partial differential equations (PDEs) [175–179]. Their execution characteristics differ markedly from the bursty, heterogeneous DL workloads discussed earlier [180].

Execution Pattern: PDE solvers are typically long-lived, tightly coupled MPI applications, where each simulation timestep alternates between compute-intensive stencil updates and communication phases—such as halo exchanges or collective reductions—resulting in a highly regular compute/communication cadence. Wall-clock durations can range from a few minutes for explicit computational fluid dynamics loops to several days for global climate ensemble runs. Job sizes scale from a single multi-GPU server to full multi-rack deployments [181].

Resource footprint: The core kernels are predominantly memory-bandwidth bound, often saturating the node-local interconnect during collective operations [182]. After the initial field data is staged, host–device transfers largely subside, making PCIe throughput a negligible factor during steady-state execution [183].

Scheduling implications: Because runtime variance is low once mesh resolution and timestep are fixed, these workloads are well suited to static bin-packing or partitioned MILP schedules that can be computed close to the utilization optimum [184]. Communication locality is equally critical: each halo exchange traverses the interconnect fabric, so job placement must minimize hop count across NVLink, NVSwitch, or InfiniBand domains—an objective effectively addressed by communication-aware MILP formulations or topology-sensitive heuristics [185,186]. Their pronounced memory-bound phases also make PDE solvers ideal candidates for energy-aware scheduling; techniques such as DVFS or slack reclamation during bandwidth-limited intervals can reduce power draw with minimal impact on time-to-solution [187].

Key takeaways. For PDE/HPC workloads, communication-aware static partitioning remains the dominant scheduling paradigm. Dynamic, learning-based approaches become advantageous only when job mixes or resource availability fluctuates rapidly—conditions rarely seen in tightly coupled production simulations.

3.7. Summary of Scheduling Algorithms and Their Applicability Concerns

We conclude by summarizing which classes of scheduling algorithms are best suited to different operating conditions and then discuss the practical challenges of deploying custom policies in real-world GPU clusters.

Applicability by scenario: The suitability of a scheduling strategy hinges on the information hierarchy, workload stochasticity, and cluster composition that prevail at deployment time. In offline deterministic settings—where every job's size and precedence constraints are known up front—exact algorithms such as MILP or DP achieve provably optimal schedules, while even well-tuned greedy heuristics can deviate sharply from the optimum. Conversely, many production clusters operate online under heavy-tailed arrival processes: DL training queues, for instance, exhibit unknown and highly skewed job sizes; here, queueing-theoretic disciplines like the Gittins index or LAS strike a principled balance between fairness and mean slowdown [74]. When the stream instead comprises predictable, quasi-periodic jobs—for example, regularly recurring inference or analytics tasks—static reservations or SJF rules already approach the optimum, with dynamic SJF closing the remaining gap whenever runtime estimates are accurate.

Heterogeneity introduces a further axis: clusters sporting multiple GPU generations benefit from throughput-normalized allocators such as Gavel [188], or from two-stage schemes that first match jobs to device classes (via ML prediction or bipartite matching) and then apply a standard scheduler within each class. In multi-tenant cloud environments, the dominant concern shifts to inter-tenant fairness and performance isolation; partial-auction mechanisms (e.g., Themis) or fair-share frameworks such as DRF are therefore layered atop latency-oriented inner loops like SJF to harmonize efficiency with service guarantees.

Finally, incorporating PDE-driven scientific workloads (Section 3.6) broadens the taxonomy without altering its conclusions. Their low runtime variance and regular all-reduce cadence make classical bin-packing and communication-aware MILP approaches nearly optimal—much like in stable, single-tenant inference farms. In contrast, reinforcement learning–based schedulers offer little advantage under such steady-state conditions, reinforcing the view that learning-based methods are most effective in bursty, multi-tenant environments.

Practical integration and operational challenges: Rolling out a bespoke GPU scheduler is ultimately an exercise in system integration: it must interoperate cleanly with the cluster's resource manager, accommodate the execution models of both distributed batch and interactive jobs, and leverage GPU-specific control APIs without disrupting user workflows. At the control-plane layer, real-time visibility into free and allocated devices is obtained through scheduler extenders or plugins—Kubernetes, for example, exposes a well-defined gRPC interface that many custom schedulers hook into [189]. Multi-node jobs add further complexity: platforms such as Slurm currently lack fine-grained GPU preemption, so any dynamic rescheduling logic must reconcile whole-node allocations across potentially hundreds of ranks. Equally important is tight coordination with deep-learning frameworks such as TensorFlow or PyTorch, whose runtime assumptions favor stable device assignments; enabling preemption therefore entails wiring checkpoint hooks and aligning interruptions with iteration boundaries to achieve "graceful" pause-and-resume semantics [190,191]. Finally, modern devices expose features—NVIDIA's MIG partitions, per-SM utilization counters, or power-limiting knobs—that a scheduler can exploit only by invoking vendor-specific APIs, adding another integration axis.

Beyond plumbing, three operational dimensions shape day-to-day viability. *Scheduler latency* is the first: sophisticated policies such as AlloX's $O(n^3)$ Hungarian matching [124] or Decima's neural inference path [127] can incur tens to hundreds of milliseconds per decision. Most production deployments therefore amortize this cost—by reevaluating every few seconds or minutes—or fall back on sub-millisecond heuristics when responsiveness trumps optimality. Second, *state movement* remains expensive; naively dumping GPU memory to disk for preemption can erase any latency gains promised by fairness-driven models. Systems thus rely on iteration-boundary checkpoints or oversubscription techniques such as SALUS, which multiplex kernels and memory to avoid full context switches at the expense of per-job throughput [192]. Third, *monitoring and adaptivity* require a fine-grained telemetry substrate—either node agents or in-band probes—that feeds live utilization and contention metrics back to the scheduler. Instrumentation that is too coarse risks stale decisions, whereas overly chatty probes can induce feedback oscillations, so designers must strike a careful balance between freshness, overhead, and stability.

Beyond per-node telemetry, multi-tenancy adds a cluster-wide layer of complexity: Multi-tenant GPU clusters bring unique scheduling challenges due to interference on shared resources. Co-locating jobs can induce contention not only on CPU cores—leading to context-switching overhead—but also on memory bandwidth, PCIe bus, and disk I/O when GPUs reside on the same node [193,194]. Distributed training workloads further exacerbate contention by saturating network links, potentially throttling performance across tenant jobs [77,195].

To mitigate these issues, modern schedulers incorporate resource topology and affinity into placement decisions. For example, awareness of NVLink connectivity or rack-level bandwidth can lead a scheduler to favor slightly less balanced but network-friendly allocations, reducing contention [131,167,196]. Parrot's coflow-aware framework dynamically adjusts bandwidth allocations during distributed machine learning, improving average job completion time by up to 18.2% in multi-tenant clusters [197].

Security and isolation are also critical. Without strong tenant separation, one job may leak information or disrupt another via GPU side channels or crashes [198]. Hardware-level isolation mechanisms such as NVIDIA's MIG partitions are often required to enforce tenant boundaries safely.

A concrete example of system-level complexity is AlloX's integration with Kubernetes. Implemented as a scheduling extender, it spanned thousands of lines of Go and Python code to manage containers, sample job runtimes on CPU versus GPU, and ultimately allocate

whole GPUs—since current clusters lack support for arbitrary GPU time-slicing [124]. This experience highlights that many theoretically optimal strategies (e.g., splitting a GPU) must be abandoned or adapted due to system limitations.

Taken together, these integration and multi-tenancy obstacles reinforce a final lesson: Successful GPU scheduling balances algorithmic elegance with engineering practicality. Designers must iterate between prototype implementation and algorithm refinement, addressing integration, monitoring, preemption, and telemetry overhead to ensure that theory translates into real-world performance improvements.

4. Looking Forward: LLM Focus

The emerging wave of GPU workloads already surpasses the capabilities of most contemporary schedulers. Training and serving billion-parameter models introduce scale, structural diversity, and dynamism that challenge classical algorithms. LLMs are a canonical stress test: they demand aggressive data, model, and pipeline parallelism, and exhibit highly variable resource footprints across execution. Yet these same characteristics—massive parallelism, bursty arrivals, and mutable throughput-latency trade-offs—also define modern HPC simulations, multi-modal generative models, and real-time analytics pipelines.

Recent systems such as Sia, Frenzy, Llumnix, Crius, Harmony, Varuna, Alpa, and TRAIL point toward four key research directions that may shape future schedulers. First, schedulers are becoming increasingly *resource-aware and learning-augmented*: analytical models are now combined with online learning to capture non-linear speedups and interference effects. Second, to close the cold-start gap we discussed for pure RL policies in Section 2.5—where new workloads suffered minutes of sub-optimal placement—emerging designs embrace *hybrid strategies* that pair fast heuristics with ML-driven micro-decisions, enabling adaptation to workload drift while maintaining worst-case performance guarantees. Third, there is a growing focus on explicitly managing *hardware heterogeneity*, as clusters mix GPU generations, interconnect fabrics, and specialized accelerators—making bandwidth, memory, and reliability as critical as raw compute. These bandwidth-aware placement schemes explicitly fill the "unsolved-constraint" column in Table 2, turning the earlier red-flagged inter-device tiering blind spot into a first-class scheduling dimension. Finally, fault tolerance and risk are being addressed as first-class optimization goals, as schedulers balance checkpointing, replication, and migration overheads in volatile environments such as spot markets or geo-distributed training —preemptively amortizing the 10–30 ms kernel-drain penalty highlighted in Section 3.5 through proactive snapshotting and live migration.

These trends surface several cross-cutting open questions: how to schedule under workload and model uncertainty; how to scale to tens of thousands of devices in multi-tenant environments; how to support emerging architectures like state-space models and mixture-of-experts; how to minimize energy and cost under power constraints; and how to decentralize the scheduling logic itself to avoid central bottlenecks. Addressing these challenges will require tighter integration of scheduling theory, performance modeling, and systems engineering.

4.1. Scaling Scheduling for Future GPU Clusters

This directly re-engages the heavy-tail mismatch documented in Section 1.5, where we showed that naive exponential assumptions can inflate 99th-percentile queueing delays by up to 4 times. It implies that LLM-serving schedulers must embed explicit tail-risk predictors. Designing queueing policies that both learn heavy-tail statistics and retain closed-form performance bounds therefore remains a high-leverage research frontier. Next-generation

deep learning platforms comprise tens of thousands of heterogeneous accelerators shared by diverse workloads, pushing legacy schedulers—designed for smaller, homogeneous fleets—well beyond their comfort zone. Modern controllers must therefore (i) scale to cluster sizes that rival supercomputers, (ii) embrace heterogeneity as a first-class dimension, and (iii) juggle multi-tenant fairness, network topology, and security in a single optimization loop. Early systems such as Sia make headway by explicitly modeling device diversity and scaling to 2000-GPU clusters, trimming average JCT by 30∼93% relative to prior heuristics [55]. Their key insight—jointly optimizing accelerator *type*, *quantity*, and *elasticity*—highlights the cost of ignoring any one axis: penalties of 40∼70% JCT when hardware heterogeneity or malleability is left out.

As jobs fan out across nodes, interconnect topology becomes equally decisive. Synchronization and sharding traffic can dominate run time unless placements are fabric-aware—co-locating tasks on NVSwitch or InfiniBand islands whenever possible. The problem generalizes classical "unrelated-parallel-machine" scheduling: the controller must select not merely how many resources to grant but which class (GPU, TPU, FPGA) under throughput and QoS targets. Extending dominant-resource or finish-time fairness to these mixed pools remains open; prototypes such as Pollux and Gavel offer initial road-maps [57,68]. At fleet scale, single-point schedulers risk bottlenecks and failures, motivating hierarchical or decentralized control planes that approximate global optima with partial information.

Network contention adds a further layer. Distributed training generates *coflows*—sets of synchronized flows whose slowest member gates progress. Coflow-aware algorithms allocate bandwidth holistically and cut JCTs by up to 61% on NVSwitch and InfiniBand-NDR fabrics [199,200]. By reasoning over coflows, these schedulers confront the 20∼40% cross-rack traffic inflation we quantified in Section 2.4. However, the empirical pathway is still blocked: no public trace today exposes per-flow affinity tags, leaving a conspicuous data void for reproducible evaluation.

Finally, multi-tenant isolation unifies security, performance, and billing accuracy. Hardware-level virtualization (e.g., NVIDIA MIG) partitions GPUs into spatial slices but may strand capacity when demand fluctuates, surfacing the very burst-induced fragmentation cost that Section 1.5 flagged as a barrier to fleet-wide efficiency. Runtime-level sandboxing enables finer time-sharing at the cost of 10∼100 ms context-switch delays—well above the sub-millisecond target of interactive DL jobs [192]. Hypervisor or enclave isolation delivers the strongest confidentiality guarantees yet restricts dynamic scaling and invites vendor lock-in [201,202]. The paucity of public traces with microsecond-level contention data leaves designers flying blind; closing this gap will require open telemetry alongside hardware-assisted, low-latency isolation primitives.

Looking ahead, schedulers for exascale GPU fleets must weave together topology-aware placement, coflow-level bandwidth management, and secure, fair multi-tenancy across heterogeneous accelerators. Whether their control planes are centralized, hierarchical, or fully decentralized, these next-generation systems will have to reconcile all three objectives at supercomputer scale—a challenge that promises rich opportunities for both system builders and scheduling theorists.

4.2. Algorithmic Challenges in Large-Scale Training Job Scheduling

Training state-of-the-art language models—with hundreds of billions of parameters—exposes scheduling challenges far beyond classical batch abstractions. Such jobs are *malleable*: they can run on widely varying numbers and types of GPUs and exhibit super-linear speed-ups up to a point of diminishing returns. Hence a scheduler must decide not only *when* and *where* to run each job, but also *how many* and *which* accelerators to allocate, revising those decisions dynamically as resources churn. Crius tackles this combinatorial

explosion by introducing *cells*—fixed pipeline-parallel configurations that discretize the search space to enable accurate performance modeling—yielding large throughput gains on heterogeneous clusters while keeping optimization tractable [203].

The parallelization strategy itself is a scheduling primitive. Hybrid data–tensor–pipeline schemes can raise per-GPU FLOPs by 20∼40% on thousand-GPU clusters, as demonstrated by Alpa on Ray for a 175 B-parameter model [204]. Jointly choosing (i) a job's internal parallel plan and (ii) its placement on unrelated parallel machines is akin to an assignment problem with exponentially many configurations; naive ILP formulations are intractable, but cell abstractions and specialized network-flow reductions show promise [203]. There is rich room for heuristics and approximation algorithms that approach optimality without exhaustive enumeration.

Elasticity further complicates matters: workloads expand or contract via batch-size or micro-batching adjustments in response to resource availability. Pollux's co-adaptive scheduling reallocates GPUs at epoch boundaries and tunes hyper-parameters on the fly [68]; Sia maintains per-job throughput curves for multiple GPU counts and solves a small optimizer each quantum [55]. A principled elasticity model therefore requires non-linear speed-up prediction across device types and counts, plus arbitration rules that prevent greedy expansions from starving neighbors—an open frontier for malleable-task theory in accelerator-rich settings.

Reliability and cost enter next. Multi-day runs on preemptible VMs must balance savings against the risk of lost progress; Varuna morphs its parallelism on the fly, re-packing when nodes vanish and exploiting arrivals to shorten makespan [205]. Conceptually, GPUs come in "cheap-but-volatile" and "stable-but-expensive" flavors; scheduling under stochastic failures then resembles stochastic project planning with checkpoint-interval tuning, erasure-coded gradient replication, or peer-to-peer state mirroring as control knobs. Researchers currently lack ground truth data needed to compare restart latency across system designs. Until that gap is addressed, it remains difficult to meaningfully benchmark how quickly poisoned or compromised jobs recover under mechanisms such as checkpointing, erasure-coded replay, or Varuna-style live repacking.

Finally, cluster scale forces a rethink of control hierarchy. A single monolithic scheduler can bottleneck at tens of thousands of GPUs. Hierarchical schemes partition capacity among frameworks or priority classes, delegating fine-grain placement to sub-schedulers, while fully decentralized approaches—where jobs make local decisions from small samples, such as in Sparrow [206]—remain largely unexplored for LLM training. Advances in volunteer/federated learning suggest that coordination-light schemes are feasible [207], but the central question is how to approximate global optimality and fairness under partial information and bounded communication.

Taken together, malleability, parallelism co-design, elasticity, reliability, and decentralized control form an intertwined agenda for both systems and theory. Addressing them will require new abstractions that couple performance-modeling fidelity with algorithmic tractability, alongside robustness techniques that scale gracefully to the next order of cluster heterogeneity.

4.3. Serving LLMs and Complex Inference Workloads: New Scheduling Frontiers

Building on the heterogeneity- and scale-oriented challenges discussed in earlier sections, we now shift focus from *training* to the equally demanding domain of production-scale *inference*. While training jobs consume more total GPU hours, LLM serving imposes far stricter real-time constraints and exposes a very different—and highly volatile—scheduling surface. Inference workloads exhibit extreme heterogeneity: queries vary in prompt length, generate an unpredictable number of output tokens, and span latency requirements from

interactive chat to offline batch scoring. Each output token triggers a full forward pass, and the GPU-resident key–value (KV) cache grows monotonically during generation [208]. As a result, request durations unfold online and are unknown *a priori*, making inference scheduling a multi-tenant analog of operating system job management under unknown runtimes.

Empirically, naive FIFO batching yields erratic latency and poor hardware utilization. Even state-of-the-art systems such as Alpa on Ray and MegaScale achieve only around 50% FLOP utilization on LLM workloads [209].

Recent work addresses this volatility along three main axes. First, *prediction-driven size-based scheduling* leverages intermediate activations to forecast the remaining runtime of each query. TRAIL, for example, reduces prediction error by 2.6× compared to prompt-only estimates and applies a lightly preemptive SRPT-style policy that halves mean latency while substantially improving 99th-percentile tail performance [210]. Second, *runtime adaptivity via migration* enables in-flight transfers of full inference sessions—including multi-GB KV caches—between GPUs. Llumnix uses this to rebalance load, defragment memory, and accelerate high-priority queries by 1.5×, reducing tail latency by up to 10× [208]. Third, at fleet scale, inference platforms must multiplex dozens of models and tenants. Static reservations waste capacity, while opportunistic sharing risks SLA violations. Emerging token-based fair schedulers manage this tradeoff by bounding per-request service time variance within 2× while sustaining high utilization [97].

When models exceed single-GPU memory, scheduling must extend to graph-level placement across devices. Helix tackles this by casting layer placement and routing as a MILP, achieving a 2.7× throughput gain over baseline heuristics [211]. These results suggest that periodic global optimization, paired with lightweight local adjustments, can deliver practical performance despite the underlying NP-hardness.

Looking ahead, four challenges dominate the inference scheduling agenda. First, *low-overhead preemption and migration* require the ability to suspend or relocate multi-GB KV caches with minimal disruption. Second, *accurate yet robust prediction* is crucial, as even modest errors can cause severe tail-latency spikes under tight SLAs. Third, *GPU memory fragmentation*, driven by variable request lengths and batch sizes, demands online repacking strategies that avoid long compaction stalls. Fourth, maintaining *steady-state stability under bursty load* will require robust admission control and autoscaling mechanisms that prevent oscillatory behavior.

Addressing these challenges will require both systems innovations—such as efficient GPU context switching, live migration, and fine-grained telemetry—and theoretical advances in queueing theory, real-time scheduling, and worst-case performance analysis. As demand for LLM inference grows, schedulers that integrate prediction, adaptive control, and rigorous guarantees will be key to building scalable, responsive, and cost-effective inference services.

4.4. Hybrid Scheduling Strategies and Learning-Augmented Algorithms

Revisiting the "Robust hybrid scheduling" row left open in Section 3, this subsection crystallizes the obstacles, payoffs, and performance targets for robust hybrid algorithms—those that preserve worst-case guarantees while still leveraging learned hints when they are accurate. Classical scheduling theory is increasingly intersecting with ML, particularly in GPU clusters, where high-dimensional job behavior defies closed-form modeling. *Learning-augmented* (or prediction-enhanced) algorithms equip online schedulers with forecasts—such as job sizes or speed-up curves—and analyze two regimes: one where predictions are accurate and another where they are adversarially wrong [212]. The key metric, the *price of misprediction*, captures how far performance can degrade when forecasts are noisy [113]. Translating this framework to accelerator-rich data centers is both natural and challenging,

as deep learning workloads interleave compute, communication, and memory stalls in patterns poorly captured by classical queueing models.

In practice, most systems adopt lightweight hybrid designs that pair heuristic search with small learned components. Sia, for example, performs fast placement enumeration and uses a throughput predictor to score candidates [55]. RL appears in early-stage prototypes—such as Google's cluster-DAG scheduler and Decima—but end-to-end RL often struggles with safety, generalization, and inference-time constraints. A more robust pattern confines learning to a well-bounded subproblem: Harmony, for instance, trains an RL agent only to select tensors for offload, while a rule-based outer loop enforces fairness and SLA compliance [213,214]. Multi-policy hybrids generalize this approach: short jobs may enter a size-based queue, while long-running training tasks are pooled under fair-share policies, echoing multi-level feedback queues. Profiling-first systems go further. Frenzy runs a short trial to estimate an LLM's execution footprint, feeds the resulting vector to a predictor, and then invokes classical optimization for placement [215]. With lightweight, systematic profiling in the first seconds of execution, this architecture could become a general design pattern.

Still, prediction-driven scheduling raises robustness concerns. Forecasting errors can cascade, leading to GPU fragmentation or inflated tail latency. Robust optimization offers a hedge—minimizing regret or optimizing high-confidence quantiles—but GPU-centric cost models (e.g., throughput versus memory fragmentation) remain underexplored. So too do learning methods that can operate within sub-second controller loops without incurring significant latency overhead.

Three research questions currently define the frontier. First, what should be learned and what should remain hard-coded? Features like speed-up curves, interference patterns, and failure probabilities offer rich signal, but may resist accurate prediction. Second, how can systems respect the tight control-loop latencies of modern clusters—often tens of milliseconds—while still benefiting from ML-driven decisions? Third, how can worst-case performance be guaranteed when predictions fail? Establishing competitive ratios or tail-latency bounds under adversarial or noisy inputs is key to ensuring that learning-augmented methods never underperform traditional heuristics.

Progress will depend on an iterative dialogue between systems practice—demonstrating real-world benefit—and algorithmic theory, which provides performance guarantees when predictions go awry. As GPU fleets grow and workload diversity increases, hybrid schedulers that blend analytic insights with judicious learning are poised to become the rule rather than the exception.

4.5. Energy Efficiency and Cost-Aware Scheduling

The escalating energy footprint of modern GPU workloads is impossible to ignore: training a single foundation model—GPT-3, for instance—has been estimated to consume several gigawatt-hours of electricity [216]. Environmental concerns and operational cost pressures therefore elevate *energy* and *cost* to first-class scheduling objectives alongside traditional metrics such as makespan, latency, and fairness. While classical HPC systems relied on coarse controls such as power capping or shifting batch jobs to align with renewable-rich intervals, deep learning clusters expose far richer knobs. Many DL tasks tolerate throughput–latency trade-offs, enabling schedulers to under-clock GPUs for non-urgent training or to steer low-accuracy inference queries to lightweight CPU models, realizing savings without violating SLOs.

Recent work confirms the promise of *budget-aware* optimization. PowerFlow, for example, learns job-specific performance–energy trade-offs over configuration spaces (GPU count, clock frequency) and minimizes JCT subject to a cluster-wide energy cap, achieving

1.5∼3.4× JCT reductions without raising total consumption [104]. Such results motivate multi-objective formulations that (i) minimize makespan under explicit energy constraints, (ii) minimize energy given per-job deadlines, or (iii) jointly optimize both. The inverse question—What is the fastest feasible schedule under a strict power envelope?—is pressing for data centers limited by delivery, cooling, or volatile electricity prices.

Inference workloads shift the emphasis from aggregate consumption to *energy per query*. Dynamic batching, request consolidation, and selective idling can concentrate load during low-traffic periods and scale out elastically as demand surges. Early CNN studies report 5% savings from adaptive batching and up to 28% when combined with DVFS [217,218]; whether similar gains extend to transformer-based LLMs remains an open question. Beyond raw usage, the *carbon intensity* of power sources now shapes scheduling: deferring non-urgent training to low-emission intervals can markedly cut footprint, provided frameworks maintain convergence under intermittent execution.

Energy and cost constraints further motivate *geo-distributed* and decentralized scheduling. Volunteer or spot-market GPUs, often heterogeneous and ephemeral, offer attractive monetary cost per FLOP but introduce churn, stragglers, and limited global visibility [219]. Meanwhile, emerging silicon supports microsecond-scale DVFS adjustments [220], yet no production schedulers fully exploit per-GPU power–performance curves or real-time telemetry, leaving a rich integration opportunity.

Collectively, these trends herald a shift toward holistic schedulers that transcend single-objective optimization. Throughput, latency, energy, fairness, and reliability interact in complex ways, and the rise of LLMs has stretched demands along every axis. Novel systems such as Frenzy (serverless heterogeneous training) [215], Llumnix (adaptive inference serving) [208], Helix (distributed fine-tuning) [211], and TRAIL (preemptive scheduling at scale) [210] each attack one facet of the broader challenge, from minimizing memory fragmentation to enabling live migration.

Important open problems persist. How can schedulers guarantee performance amid uncertainty in job profiles, resource availability, and energy pricing? What are the theoretical utilization limits under heavy-tailed workloads and heterogeneous hardware? How should systems remain robust to tail events such as correlated stragglers or sudden spot revocations? As GPU fleets approach exascale and serve an increasingly diverse model mix, answers to these questions will be pivotal. Taken together, these questions re-activate the "Energy-aware scheduling" row that Table 8 left marked open. By tracing the narrative arc from classical heuristics (Section 2) to learning-augmented hybrids (Section 3), we make clear that volatile energy prices now bound the feasible design space and sharpen the theoretical limits for any next-generation GPU scheduler.

To ground those gaps in real deployments, Table 7 closes a remaining context void: it cross-maps each challenge onto four arenas—LLM training, LLM inference, multi-tenant clouds, and edge/IoT clusters—and annotates every cell with a rough "solution-maturity" score, highlighting where research effort is most urgently needed.

Looking ahead, we foresee a convergence of ideas from operating systems, distributed systems, economics, and machine learning. Hybrid schedulers will blend multi-objective optimization, auction-based resource negotiation, and learning-augmented heuristics to maximize both *accuracy per dollar* and *work per joule* while respecting stringent latency SLAs. By embracing the complexity of modern hardware, models, and objectives, next-generation GPU schedulers can evolve in lock-step with the frontier of AI research and hardware innovation.

Table 7. Contextual relevance of open GPU-scheduling challenges across deployment scenarios.

Open Challenge	LLM Training	LLM Inference	Mixed-Tenant Clusters	Edge/IoT	Solution Maturity
Coflow-aware scheduling	High	Medium	High	Low	Low
Secure multi-tenant isolation	Medium	High	High	Medium	Medium
Energy-aware DVFS scheduling	Medium	Medium	High	High	High
Topology-aware placement for trillion-parameter LLMs	High	Medium	Medium	Low	Medium
Cross-objective (carbon/cost/fairness) schedulers	Medium	Low	High	Low	Low

5. Conclusions

This survey has traced four algorithmic lineages—greedy/heuristic packing, exact optimization, queue-theoretic policies, and learning-augmented hybrids—and highlighted that the most effective GPU schedulers emerge when insights from all four converge. Yet the map is not the territory: each lineage relies on workload models and benchmarks that are increasingly outdated for today's hyperscale, multi-tenant clusters. Real progress now depends on better measurements. To that end, we outline five actionable open problems, each linked to a specific quantitative gap in existing public benchmarks, as summarized in Table 8.

Table 8. Quantitative blind spots currently blocking progress in GPU scheduling research.

Open Problems	Missing Benchmark Signal	Why It Matters
Coflow-aware scheduling for ultra-fast fabrics	No public trace records per-flow bytes and timing on NVSwitch or InfiniBand-NDR clusters.	Would enable direct evaluation of coflow algorithms on real fabrics, guiding both network stack tuning and scheduler design (see also Parrot's 18% JCT improvement in Section 3).
Secure multi-tenant GPU isolation	No microbenchmark suite reports cache/SM side-channel leakage across MIG or comparable partitions under production drivers.	Establishes the empirical basis for certifying isolation guarantees and designing schedulers that safely colocate untrusted tenants.
Energy-aware DVFS under tail-latency SLOs	DVFS studies typically report only median (p50) inference latency; tail latencies (p90–p99) under load remain unmeasured.	Clarifies the true energy–latency trade-off, enabling operators to meet service-level objectives while minimizing power consumption. Appendix A shows that even a cubic power model cannot predict p99 under bursty inference.
Topology-aware placement for trillion-parameter LLMs	No open trace logs tensor-parallel bandwidth or collective communication latency beyond 16-GPU slices.	Provides ground truth for placement algorithms to minimize communication hotspots in large-model training.
Cross-objective schedulers (cost × carbon × fairness)	Benchmarks lack real-time electricity pricing, carbon intensity, and per-user slowdown metrics in a single trace.	Enables Pareto-optimal scheduler design that balances budget, sustainability, and equity goals.

Releasing the five datasets outlined above would shift the field from qualitative debates to quantitative "horse races," enabling the community to report *regression-style*

improvements rather than isolated case studies. Without such data, advances in areas like coflow scheduling, DVFS, or carbon-aware bidding will remain anecdotal and difficult to reproduce.

Ultimately, effective GPU scheduling is a problem of systems synthesis—combining elegant algorithms with the practical complexities of drivers, containers, telemetry, and multi-tenant isolation. This synthesis now requires an empirical foundation that reflects the scale, security, and sustainability imperatives of modern AI workloads. Building that foundation is the field's most urgent and unifying challenge.

Author Contributions: Conceptualization, R.C., F.L. and S.S.; methodology, R.C., F.L. and S.S.; validation, R.C., F.L. and S.S.; formal analysis, F.L.; resources, R.C., F.L. and S.S.; data curation, R.C., F.L. and S.S.; writing—original draft preparation, F.L. and R.C.; writing—review and editing, R.C., F.L. and S.S.; visualization, R.C., F.L. and S.S.; supervision, F.L. and S.S.; project administration, F.L. and S.S.; funding acquisition, F.L. and S.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

List of abbreviations used in this paper:

Abbreviations	Meaning/Context
ASIC	Application-Specific Integrated Circuit (fixed-function chip)
CPU	Central Processing Unit (general-purpose core)
CP-SAT	Constraint Programming—Satisfiability (OR-Tools solver)
DL	Deep Learning (neural-network workloads)
DP	Dynamic Programming (exact but exponential-time algorithm)
DRAM	Dynamic Random-Access Memory (commodity main memory)
DRF	Dominant Resource Fairness (a multi-resource fair sharing policy)
DVFS	Dynamic Voltage and Frequency Scaling (power-saving knob)
FPGA	Field-Programmable Gate Array (reconfigurable device)
GPU	Graphics Processing Unit (parallel accelerator)
gRPC	Google Remote Procedure Call (GPU resource manager)
HBM	High-Bandwidth Memory (stacked on-package DRAM)
HPC	High-Performance Computing (science & engineering jobs)
ILP	Integer Linear Programming (exact optimization formulation)
JCT	Job Completion Time (arrival $\rightarrow$ finish interval)
KV	Key-Value cache in LLMs to store previously computed attention keys and values to avoid redundant calculations
LLM	Large Language Model (GPT-class neural networks)
MIG	Multi-Instance GPU (NVIDIA partitioning feature)
MILP	Mixed-Integer Linear Programming (ILP with continuous variables)
ML	Machine Learning (broader umbrella including DL)
NUMA	Non-Uniform Memory Access (multi-socket memory topology)
NVLink	NVIDIA high-speed point-to-point GPU interconnect (40 $\sim$ 600GB/s per link)
NVML	Nvidia Management Library (API for monitoring and managing Nvidia GPUs)
NVSwitch	On-node crossbar switch that connects multiple NVLinks
PCIe	Peripheral Component Interconnect Express
PDE	Partial Differential Equation (scientific HPC workloads)
QoS	Quality of Service (aggregate performance target)

RAPL	Running Average Power Limit scheduling policy
RL	Reinforcement Learning (data-driven scheduling paradigm)
SJF	Shortest-Job-First (greedy scheduling heuristic)
SLO	Service-Level Objective (numerical QoS target, e.g., p95 latency)
SM	Streaming Multiprocessor (GPU core cluster)
SMT	Simultaneous Multithreading (CPU core sharing)
SRPT	Shortest Remaining Processing Time scheduling policy
SSM	Streaming SM (informal shorthand for a single GPU SM)
TDP	Thermal Design Power (maximum sustained power a cooling solution is designed to dissipate)
TPU	Tensor Processing Unit (Google ASIC for ML)
VRAM	Video Random-Access Memory (legacy term for GPU memory)

Appendix A. Per-Job Energy Consumption Model

We denote by E_j the total energy consumed by job j:

$$E_j = \int_0^{T_j} P_{\text{GPU}}(t)dt,$$

where T_j is the job's makespan and P_{GPU} the instantaneous device power.

Building on the findings of Albers [221], dynamic power consumption scales approximately with the cube of the SM frequency—a relationship that has also been observed in GPUs [222].

$$P_{\text{dyn}} = \kappa \cdot f^3,$$

where κ is a device-specific coefficient calibrated once via NVML (Nvidia management library) power sampling in the P0 state (typical values: $\kappa = 1.2{\sim}1.4$ W/Hz^3 for an A100 40 GB at 250 W TDP (thermal design power), the maximum heat a processor is designed to dissipate).

Han et al. [222] also fit DNN inference latency to

$$t(f) = a \cdot f^{-b} + c, \tag{A1}$$

with $b \approx 1$ for compute-bound kernels and $b < 1$ when memory bandwidth dominates. Combining (A1) with the cubic power law gives

$$E_j(f) = \kappa \cdot f^3 \left(a \cdot f^{-b} + c\right) = \kappa \cdot a \cdot f^{3-b} + \kappa \cdot c \cdot f^3. \tag{A2}$$

For autoregressive LLM workloads, Kakolyris et al. [223] show that *iteration-level* DVFS guided by a polynomial tail-latency predictor can cut energy by 22.8∼45.5% on A100/A30 GPUs while meeting an 8.4∼10 p99 latency SLO. Their scheme is compatible with Equation (A2): the scheduler simply selects the minimum frequency f that satisfies the predicted latency bound.

A cubic-frequency power model, calibrated empirically and coupled with the latency fit of Equation (A1), offers sufficient fidelity for cluster-level optimization while remaining analytically tractable.

References

1. Dally, W.J.; Keckler, S.W.; Kirk, D.B. Evolution of the graphics processing unit (GPU). *IEEE Micro* **2021**, *41*, 42–51. [CrossRef]
2. Peddie, J. *The History of the GPU-Steps to Invention*; Springer: Berlin/Heidelberg, Germany, 2023.
3. Peddie, J. What is a GPU? In *The History of the GPU-Steps to Invention*; Springer: Berlin/Heidelberg, Germany, 2023; pp. 333–345.

4. Cano, A. A survey on graphic processing unit computing for large-scale data mining. *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.* **2018**, *8*, e1232. [CrossRef]
5. Shankar, S. Energy Estimates Across Layers of Computing: From Devices to Large-Scale Applications in Machine Learning for Natural Language Processing, Scientific Computing, and Cryptocurrency Mining. In Proceedings of the 2023 IEEE High Performance Extreme Computing Conference (HPEC), Boston, MA, USA, 25–29 September 2023; pp. 1–6.
6. Hou, Q.; Qiu, C.; Mu, K.; Qi, Q.; Lu, Y. A cloud gaming system based on NVIDIA GRID GPU. In Proceedings of the 2014 13th International Symposium on Distributed Computing and Applications to Business, Engineering and Science, Xianning, China, 24–27 November 2014; pp. 73–77.
7. Pathania, A.; Jiao, Q.; Prakash, A.; Mitra, T. Integrated CPU-GPU power management for 3D mobile games. In Proceedings of the 51st Annual Design Automation Conference, San Francisco, CA, USA, 1–5 June 2014; pp. 1–6.
8. Mills, N.; Mills, E. Taming the energy use of gaming computers. *Energy Effic.* **2016**, *9*, 321–338. [CrossRef]
9. Teske, D. *NVIDIA Corporation: A Strategic Audit*; University of Nebraska-Lincoln: Lincoln, NE, USA, 2018.
10. Moya, V.; Gonzalez, C.; Roca, J.; Fernandez, A.; Espasa, R. Shader performance analysis on a modern GPU architecture. In Proceedings of the 38th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'05), Barcelona, Spain, 12–16 November 2005; pp. 10–364.
11. Kirk, D. NVIDIA CUDA software and GPU parallel computing architecture. In Proceedings of the International Symposium on Memory Management (ISMM), Montreal, QC, Canada, 21–22 October 2007; Volume 7, pp. 103–104.
12. Peddie, J. Mobile GPUs. In *The History of the GPU-New Developments*; Springer: Berlin/Heidelberg, Germany, 2023; pp. 101–185.
13. Gera, P.; Kim, H.; Kim, H.; Hong, S.; George, V.; Luk, C.K. Performance characterisation and simulation of Intel's integrated GPU architecture. In Proceedings of the 2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), Belfast, UK, 2–4 April 2018; pp. 139–148.
14. Rajagopalan, G.; Thistle, J.; Polzin, W. The potential of GPU computing for design in RotCFD. In Proceedings of the AHS Technical Meeting on Aeromechanics Design for Transformative Vertical Flight, San Francisco, CA, USA, 16–18 January 2018.
15. McClanahan, C. History and evolution of GPU architecture. *Surv. Pap.* **2010**, *9*, 1–7.
16. Lee, V.W.; Kim, C.; Chhugani, J.; Deisher, M.; Kim, D.; Nguyen, A.D.; Satish, N.; Smelyanskiy, M.; Chennupaty, S.; Hammarlund, P.; et al. Debunking the 100X GPU vs. CPU myth: An evaluation of throughput computing on CPU and GPU. In Proceedings of the 37th Annual International Symposium on Computer Architecture, Saint-Malo, France, 19–23 June 2010; pp. 451–460.
17. Bergstrom, L.; Reppy, J. Nested data-parallelism on the GPU. In Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming, Copenhagen, Denmark, 10–12 September 2012; pp. 247–258.
18. Thomas, W.; Daruwala, R.D. Performance comparison of CPU and GPU on a discrete heterogeneous architecture. In Proceedings of the 2014 International Conference on Circuits, Systems, Communication and Information Technology Applications (CSCITA), Mumbai, India, 4–5 April 2014; pp. 271–276.
19. Svedin, M.; Chien, S.W.; Chikafa, G.; Jansson, N.; Podobas, A. Benchmarking the Nvidia GPU lineage: From early K80 to modern A100 with asynchronous memory transfers. In Proceedings of the 11th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies, Online, 21–23 June 2021; pp. 1–6.
20. Bhargava, R.; Troester, K. AMD next generation "Zen 4" core and 4th gen AMD EPYC server CPUs. *IEEE Micro* **2024**, *44*, 8–17. [CrossRef]
21. Hill, M.D.; Marty, M.R. Amdahl's law in the multicore era. *Computer* **2008**, *41*, 33–38. [CrossRef]
22. Rubio, J.; Bilbao, C.; Saez, J.C.; Prieto-Matias, M. Exploiting elasticity via OS-runtime cooperation to improve CPU utilization in multicore systems. In Proceedings of the 2024 32nd Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP), Dublin, Ireland, 20–22 March 2024; pp. 35–43.
23. Jones, C.; Gartung, P. CMSSW Scaling Limits on Many-Core Machines. *arXiv* **2023**, arXiv:2310.02872.
24. Gorman, M.; Engineer, S.K.; Jambor, M. Optimizing Linux for AMD EPYC 7002 Series Processors with SUSE Linux Enterprise 15 SP1. In *SUSE Best Practices*; SUSE: Nuremberg, Germany, 2019.
25. Fan, Z.; Qiu, F.; Kaufman, A.; Yoakum-Stover, S. GPU cluster for high performance computing. In Proceedings of the SC'04: Proceedings of the 2004 ACM/IEEE Conference on Supercomputing, Pittsburgh, PA, USA, 6–12 November 2004; p. 47.
26. Kimm, H.; Paik, I.; Kimm, H. Performance comparision of TPU, GPU, CPU on Google colaboratory over distributed deep learning. In Proceedings of the 2021 IEEE 14th International Symposium on Embedded Multicore/Many-Core Systems-on-Chip (MCSoC), Singapore, 20–23 December 2021; pp. 312–319.
27. Wang, Y.E.; Wei, G.Y.; Brooks, D. Benchmarking TPU, GPU, and CPU platforms for deep learning. *arXiv* **2019**, arXiv:1907.10701.
28. Narayanan, D.; Shoeybi, M.; Casper, J.; LeGresley, P.; Patwary, M.; Korthikanti, V.; Vainbrand, D.; Kashinkunti, P.; Bernauer, J.; Catanzaro, B.; et al. Efficient large-scale language model training on GPU clusters using megatron-lm. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, St. Louis, MO, USA, 14–19 November 2021; pp. 1–15.

29. Palacios, J.; Triska, J. *A Comparison of Modern GPU and CPU Architectures: And the Common Convergence of Both*; Oregon State University: Corvallis, OR, USA, 2011.
30. Haugen, P.; Myers, I.; Sadler, B.; Whidden, J. A Basic Overview of Commonly Encountered types of Random Access Memory (RAM). Class Notes of Computer Architecture II. Rose-Hulman Institute of Technology, Terre Haute, IN, USA. Available online: https://www.docsity.com/en/docs/basic-types-of-random-access-memory-lecture-notes-ece-332/6874965/ (accessed on 24 April 2025).
31. Kato, S.; McThrow, M.; Maltzahn, C.; Brandt, S. Gdev: First-Class GPU Resource Management in the Operating System. In Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC 12), Boston, MA, USA, 13–15 June 2012; pp. 401–412.
32. Kato, S.; Brandt, S.; Ishikawa, Y.; Rajkumar, R. Operating systems challenges for GPU resource management. In Proceedings of the International Workshop on Operating Systems Platforms for Embedded Real-Time Applications, Porto, Portugal, 5 July 2011; pp. 23–32.
33. Wen, Y.; O'Boyle, M.F. Merge or separate? Multi-job scheduling for OpenCL kernels on CPU/GPU platforms. In *Proceedings of the General Purpose GPUs*; Association for Computing Machinery: New York, NY, USA, 2017; pp. 22–31.
34. Tu, C.H.; Lin, T.S. Augmenting operating systems with OpenCL accelerators. *ACM Trans. Des. Autom. Electron. Syst. (TODAES)* **2019**, *24*, 1–29. [CrossRef]
35. Chazapis, A.; Nikolaidis, F.; Marazakis, M.; Bilas, A. Running kubernetes workloads on HPC. In Proceedings of the International Conference on High Performance Computing, Denver, CO, USA, 11–17 November 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 181–192.
36. Weng, Q.; Yang, L.; Yu, Y.; Wang, W.; Tang, X.; Yang, G.; Zhang, L. Beware of Fragmentation: Scheduling GPU-Sharing Workloads with Fragmentation Gradient Descent. In Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC 23), Boston, MA, USA, 10–12 July 2023; pp. 995–1008.
37. Kenny, J.; Knight, S. *Kubernetes for HPC Administration*; Technical Report; Sandia National Lab. (SNL-NM): Albuquerque, NM, USA, 2021.
38. Burns, B.; Grant, B.; Oppenheimer, D.; Brewer, E.; Wilkes, J. Borg, Omega, and Kubernetes: Lessons learned from three container-management systems over a decade. *Queue* **2016**, *14*, 70–93. [CrossRef]
39. Vavilapalli, V.K.; Murthy, A.C.; Douglas, C.; Agarwal, S.; Konar, M.; Evans, R.; Graves, T.; Lowe, J.; Shah, H.; Seth, S.; et al. Apache Hadoop Yarn: Yet another resource negotiator. In Proceedings of the 4th Annual Symposium on Cloud Computing, Seattle, WA, USA, 3–5 November 2013; pp. 1–16.
40. Kato, S.; Lakshmanan, K.; Rajkumar, R.; Ishikawa, Y. TimeGraph: GPU Scheduling for Real-Time Multi-Tasking Environments. In Proceedings of the 2011 USENIX Annual Technical Conference (USENIX ATC 11), Portland, OR, USA, 15–17 June 2011.
41. Duato, J.; Pena, A.J.; Silla, F.; Mayo, R.; Quintana-Ortí, E.S. rCUDA: Reducing the number of GPU-based accelerators in high performance clusters. In Proceedings of the 2010 International Conference on High Performance Computing & Simulation, Caen, France, 28 June–2 July 2010; pp. 224–231.
42. Agrawal, A.; Mueller, S.M.; Fleischer, B.M.; Sun, X.; Wang, N.; Choi, J.; Gopalakrishnan, K. DLFloat: A 16-b floating point format designed for deep learning training and inference. In Proceedings of the 2019 IEEE 26th Symposium on Computer Arithmetic (ARITH), Kyoto, Japan, 10–12 June 2019; pp. 92–95.
43. Yeung, G.; Borowiec, D.; Friday, A.; Harper, R.; Garraghan, P. Towards GPU utilization prediction for cloud deep learning. In Proceedings of the 12th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 20), Boston, MA, USA, 13 July 2020.
44. Jeon, M.; Venkataraman, S.; Phanishayee, A.; Qian, J.; Xiao, W.; Yang, F. Analysis of Large-Scale Multi-Tenant GPU clusters for DNN training workloads. In Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC 19), Renton, WA, USA, 10–12 July 2019; pp. 947–960.
45. Wu, G.; Greathouse, J.L.; Lyashevsky, A.; Jayasena, N.; Chiou, D. GPGPU performance and power estimation using machine learning. In Proceedings of the 2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA), Burlingame, CA, USA, 7–11 February 2015; pp. 564–576.
46. Boutros, A.; Nurvitadhi, E.; Ma, R.; Gribok, S.; Zhao, Z.; Hoe, J.C.; Betz, V.; Langhammer, M. Beyond peak performance: Comparing the real performance of AI-optimized FPGAs and GPUs. In Proceedings of the 2020 International Conference on Field-Programmable technology (ICFPT), Maui, HI, USA, 9–11 December 2020; pp. 10–19.
47. Nordmark, R.; Olsén, T. A Ray Tracing Implementation Performance Comparison between the CPU and the GPU. Bachelor Thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2022.
48. Sun, Y.; Agostini, N.B.; Dong, S.; Kaeli, D. Summarizing CPU and GPU design trends with product data. *arXiv* **2019**, arXiv:1911.11313.
49. Li, C.; Sun, Y.; Jin, L.; Xu, L.; Cao, Z.; Fan, P.; Kaeli, D.; Ma, S.; Guo, Y.; Yang, J. Priority-based PCIe scheduling for multi-tenant multi-GPU systems. *IEEE Comput. Archit. Lett.* **2019**, *18*, 157–160. [CrossRef]
50. Chopra, B. Enhancing Machine Learning Performance: The Role of GPU-Based AI Compute Architectures. *J. Knowl. Learn. Sci. Technol.* **2024**, *6386*, 29–42. [CrossRef]

51. Baker, M.; Buyya, R. Cluster computing at a glance. *High Perform. Clust. Comput. Archit. Syst.* **1999**, *1*, 12.
52. Jararweh, Y.; Hariri, S. Power and performance management of GPUs based cluster. *Int. J. Cloud Appl. Comput. (IJCAC)* **2012**, *2*, 16–31. [CrossRef]
53. Wesolowski, L.; Acun, B.; Andrei, V.; Aziz, A.; Dankel, G.; Gregg, C.; Meng, X.; Meurillon, C.; Sheahan, D.; Tian, L.; et al. Datacenter-scale analysis and optimization of GPU machine learning workloads. *IEEE Micro* **2021**, *41*, 101–112. [CrossRef]
54. Kindratenko, V.V.; Enos, J.J.; Shi, G.; Showerman, M.T.; Arnold, G.W.; Stone, J.E.; Phillips, J.C.; Hwu, W.m. GPU clusters for high-performance computing. In Proceedings of the 2009 IEEE International Conference on Cluster Computing and Workshops, New Orleans, LA, USA, 31 August–4 September 2009; pp. 1–8.
55. Jayaram Subramanya, S.; Arfeen, D.; Lin, S.; Qiao, A.; Jia, Z.; Ganger, G.R. Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. In Proceedings of the 29th Symposium on Operating Systems Principles, Koblenz, Germany, 23–26 October 2023; pp. 642–657.
56. Xiao, W.; Bhardwaj, R.; Ramjee, R.; Sivathanu, M.; Kwatra, N.; Han, Z.; Patel, P.; Peng, X.; Zhao, H.; Zhang, Q.; et al. Gandiva: Introspective cluster scheduling for deep learning. In Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), Carlsbad, CA, USA, 8–10 October 2018; pp. 595–610.
57. Narayanan, D.; Santhanam, K.; Kazhamiaka, F.; Phanishayee, A.; Zaharia, M. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), Online, 4–6 November 2020; pp. 481–498.
58. Li, A.; Song, S.L.; Chen, J.; Li, J.; Liu, X.; Tallent, N.R.; Barker, K.J. Evaluating modern GPU interconnect: PCIe, NVLink, NV-SLI, NVSwitch and GPUDirect. *IEEE Trans. Parallel Distrib. Syst.* **2019**, *31*, 94–110. [CrossRef]
59. Kousha, P.; Ramesh, B.; Suresh, K.K.; Chu, C.H.; Jain, A.; Sarkauskas, N.; Subramoni, H.; Panda, D.K. Designing a profiling and visualization tool for scalable and in-depth analysis of high-performance GPU clusters. In Proceedings of the 2019 IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC), Hyderabad, India, 17–20 December 2019; pp. 93–102.
60. Liao, C.; Sun, M.; Yang, Z.; Xie, J.; Chen, K.; Yuan, B.; Wu, F.; Wang, Z. LoHan: Low-Cost High-Performance Framework to Fine-Tune 100B Model on a Consumer GPU. *arXiv* **2024**, arXiv:2403.06504.
61. Isaev, M.; McDonald, N.; Vuduc, R. Scaling infrastructure to support multi-trillion parameter LLM training. In Proceedings of the Architecture and System Support for Transformer Models (ASSYST@ ISCA 2023), Oralndo, FL, USA, 17–21 June 2023.
62. Weng, Q.; Xiao, W.; Yu, Y.; Wang, W.; Wang, C.; He, J.; Li, Y.; Zhang, L.; Lin, W.; Ding, Y. MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. In Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22), Renton, WA, USA, 4–6 April 2022; pp. 945–960.
63. Kumar, A.; Subramanian, K.; Venkataraman, S.; Akella, A. Doing more by doing less: How structured partial backpropagation improves deep learning clusters. In Proceedings of the 2nd ACM International Workshop on Distributed Machine Learning, Virtual, 7 December 2021; pp. 15–21.
64. Hu, Q.; Sun, P.; Yan, S.; Wen, Y.; Zhang, T. Characterization and prediction of deep learning workloads in large-scale GPU datacenters. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, St. Louis, MO, USA, 14–19 November 2021; pp. 1–15.
65. Crankshaw, D.; Wang, X.; Zhou, G.; Franklin, M.J.; Gonzalez, J.E.; Stoica, I. Clipper: A low-latency online prediction serving system. In Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17), Boston, MA, USA, 27–29 March 2017; pp. 613–627.
66. Peng, Y.; Bao, Y.; Chen, Y.; Wu, C.; Guo, C. Optimus: An efficient dynamic resource scheduler for deep learning clusters. In Proceedings of the Thirteenth EuroSys Conference, Porto, Portugal, 23–26 April 2018; pp. 1–14.
67. Yu, M.; Tian, Y.; Ji, B.; Wu, C.; Rajan, H.; Liu, J. Gadget: Online resource optimization for scheduling ring-all-reduce learning jobs. In Proceedings of the IEEE INFOCOM 2022-IEEE Conference on Computer Communications, Virtual, 2–5 May 2022; pp. 1569–1578.
68. Qiao, A.; Choe, S.K.; Subramanya, S.J.; Neiswanger, W.; Ho, Q.; Zhang, H.; Ganger, G.R.; Xing, E.P. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation OSDI 21), Online, 14–16 July 2021.
69. Zhang, Z.; Zhao, Y.; Liu, J. Octopus: SLO-aware progressive inference serving via deep reinforcement learning in multi-tenant edge cluster. In Proceedings of the International Conference on Service-Oriented Computing, Rome, Italy, 28 November–1 December 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 242–258.
70. Chaudhary, S.; Ramjee, R.; Sivathanu, M.; Kwatra, N.; Viswanatha, S. Balancing efficiency and fairness in heterogeneous GPU clusters for deep learning. In Proceedings of the Fifteenth European Conference on Computer Systems, Heraklion, Greece, 27–30 April 2020; pp. 1–16.
71. Pinedo, M.L. *Scheduling: Theory, Algorithms, and Systems*, 6th ed.; Springer: Berlin/Heidelberg, Germany, 2022.

72. Shao, J.; Ma, J.; Li, Y.; An, B.; Cao, D. GPU scheduling for short tasks in private cloud. In Proceedings of the 2019 IEEE International Conference on Service-Oriented System Engineering (SOSE), San Francisco, CA, USA, 4–9 April 2019; pp. 215–2155.
73. Han, M.; Zhang, H.; Chen, R.; Chen, H. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), Carlsbad, CA, USA, 11–13 July 2022; pp. 539–558.
74. Gu, J.; Chowdhury, M.; Shin, K.G.; Zhu, Y.; Jeon, M.; Qian, J.; Liu, H.; Guo, C. Tiresias: A GPU cluster manager for distributed deep learning. In Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19), Boston, MA, USA, 26–28 February 2019; pp. 485–500.
75. Memarzia, P.; Ray, S.; Bhavsar, V.C. The art of efficient in-memory query processing on NUMA systems: A systematic approach. In Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE), Dallas, TX, USA, 20–24 April 2020; pp. 781–792.
76. Vilestad, J. An Evaluation of GPU Virtualization. Degree Thesis, Luleå University of Technology, Luleå, Sweden, 2024.
77. Amaral, M.; Polo, J.; Carrera, D.; Seelam, S.; Steinder, M. Topology-aware GPU scheduling for learning workloads in cloud environments. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, Denver, CO, USA, 12–17 November 2017; pp. 1–12.
78. Zhao, Y.; Liu, Y.; Peng, Y.; Zhu, Y.; Liu, X.; Jin, X. Multi-resource interleaving for deep learning training. In Proceedings of the ACM SIGCOMM 2022 Conference, Amsterdam, The Netherlands, 22–26 August 2022; pp. 428–440.
79. Mohan, J.; Phanishayee, A.; Kulkarni, J.; Chidambaram, V. Looking beyond GPUs for DNN scheduling on {Multi-Tenant} clusters. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), Carlsbad, CA, USA, 11–13 July 2022; pp. 579–596.
80. Reuther, A.; Byun, C.; Arcand, W.; Bestor, D.; Bergeron, B.; Hubbell, M.; Jones, M.; Michaleas, P.; Prout, A.; Rosa, A.; et al. Scalable system scheduling for HPC and big data. *J. Parallel Distrib. Comput.* **2018**, *111*, 76–92. [CrossRef]
81. Ye, Z.; Sun, P.; Gao, W.; Zhang, T.; Wang, X.; Yan, S.; Luo, Y. Astraea: A fair deep learning scheduler for multi-tenant GPU clusters. *IEEE Trans. Parallel Distrib. Syst.* **2021**, *33*, 2781–2793. [CrossRef]
82. Mao, H.; Alizadeh, M.; Menache, I.; Kandula, S. Resource management with deep reinforcement learning. In Proceedings of the 15th ACM Workshop on Hot Topics in Networks, Atlanta, GA, USA, 9–10 November 2016; pp. 50–56.
83. Feitelson, D.G.; Rudolph, L.; Schwiegelshohn, U.; Sevcik, K.C.; Wong, P. Theory and practice in parallel job scheduling. In *Proceedings of the Job Scheduling Strategies for Parallel Processing: IPPS'97 Processing Workshop, Geneva, Switzerland, 5 April 1997*; Proceedings 3; Springer: Berlin/Heidelberg, Germany, 1997; pp. 1–34.
84. Gao, W.; Ye, Z.; Sun, P.; Wen, Y.; Zhang, T. Chronus: A novel deadline-aware scheduler for deep learning training jobs. In Proceedings of the ACM Symposium on Cloud Computing, Seattle, WA, USA, 1–4 November 2021; pp. 609–623.
85. Mahajan, K.; Balasubramanian, A.; Singhvi, A.; Venkataraman, S.; Akella, A.; Phanishayee, A.; Chawla, S. Themis: Fair and efficient GPU cluster scheduling. In Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), Santa Clara, CA, USA, 25–27 February 2020; pp. 289–304.
86. Lin, C.Y.; Yeh, T.A.; Chou, J. DRAGON: A Dynamic Scheduling and Scaling Controller for Managing Distributed Deep Learning Jobs in Kubernetes Cluster. In Proceedings of the International Conference on Cloud Computing and Services Science (CLOSER), Heraklion, Greece, 2–4 May 2019; pp. 569–577.
87. Bian, Z.; Li, S.; Wang, W.; You, Y. Online evolutionary batch size orchestration for scheduling deep learning workloads in GPU clusters. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, St. Louis, MO, USA, 14–19 November 2021; pp. 1–15.
88. Wang, Q.; Shi, S.; Wang, C.; Chu, X. Communication contention aware scheduling of multiple deep learning training jobs. *arXiv* **2020**, arXiv:2002.10105.
89. Rajasekaran, S.; Ghobadi, M.; Akella, A. CASSINI: Network-Aware Job Scheduling in Machine Learning Clusters. In Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), Santa Clara, CA, USA, 16–18 April 2024; pp. 1403–1420.
90. Yeung, G.; Borowiec, D.; Yang, R.; Friday, A.; Harper, R.; Garraghan, P. Horus: Interference-aware and prediction-based scheduling in deep learning systems. *IEEE Trans. Parallel Distrib. Syst.* **2021**, *33*, 88–100. [CrossRef]
91. Garg, S.; Kothapalli, K.; Purini, S. Share-a-GPU: Providing simple and effective time-sharing on GPUs. In Proceedings of the 2018 IEEE 25th International Conference on High Performance Computing (HiPC), Bengaluru, India, 17–20 December 2018; pp. 294–303.
92. Kubiak, W.; van de Velde, S. Scheduling deteriorating jobs to minimize makespan. *Nav. Res. Logist. (NRL)* **1998**, *45*, 511–523. [CrossRef]
93. Mokoto, E. Scheduling to minimize the makespan on identical parallel Machines: An LP-based algorithm. *Investig. Oper.* **1999**, *8*, 97–107.

94. Kononov, A.; Gawiejnowicz, S. NP-hard cases in scheduling deteriorating jobs on dedicated machines. *J. Oper. Res. Soc.* **2001**, *52*, 708–717. [CrossRef]
95. Cao, J.; Guan, Y.; Qian, K.; Gao, J.; Xiao, W.; Dong, J.; Fu, B.; Cai, D.; Zhai, E. Crux: GPU-efficient communication scheduling for deep learning training. In Proceedings of the ACM SIGCOMM 2024 Conference, Sydney, Australia, 4–8 August 2024; pp. 1–15.
96. Zhong, J.; He, B. Kernelet: High-throughput GPU kernel executions with dynamic slicing and scheduling. *IEEE Trans. Parallel Distrib. Syst.* **2013**, *25*, 1522–1532. [CrossRef]
97. Sheng, Y.; Cao, S.; Li, D.; Zhu, B.; Li, Z.; Zhuo, D.; Gonzalez, J.E.; Stoica, I. Fairness in serving large language models. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), Santa Clara CA USA, 10–12 July 2024; pp. 965–988.
98. Ghodsi, A.; Zaharia, M.; Hindman, B.; Konwinski, A.; Shenker, S.; Stoica, I. Dominant resource fairness: Fair allocation of multiple resource types. In Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11), Boston, MA, USA, 30 March–1 April 2011.
99. Sun, P.; Wen, Y.; Ta, N.B.D.; Yan, S. Towards distributed machine learning in shared clusters: A dynamically-partitioned approach. In Proceedings of the 2017 IEEE International Conference on Smart Computing (SMARTCOMP), Hong Kong, China, 29–31 May 2017; pp. 1–6.
100. Mei, X.; Chu, X.; Liu, H.; Leung, Y.W.; Li, Z. Energy efficient real-time task scheduling on CPU-GPU hybrid clusters. In Proceedings of the IEEE INFOCOM 2017-IEEE Conference on Computer Communications, Atlanta, GA, USA, 1–4 May 2017; pp. 1–9.
101. Guerreiro, J.; Ilic, A.; Roma, N.; Tomas, P. GPGPU power modeling for multi-domain voltage-frequency scaling. In Proceedings of the 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA), Vienna, Austria, 24–28 February 2018; pp. 789–800.
102. Wang, Q.; Chu, X. GPGPU performance estimation with core and memory frequency scaling. *IEEE Trans. Parallel Distrib. Syst.* **2020**, *31*, 2865–2881. [CrossRef]
103. Ge, R.; Vogt, R.; Majumder, J.; Alam, A.; Burtscher, M.; Zong, Z. Effects of dynamic voltage and frequency scaling on a k20 GPU. In Proceedings of the 2013 42nd International Conference on Parallel Processing, Lyon, France, 1–4 October 2013; pp. 826–833.
104. Gu, D.; Xie, X.; Huang, G.; Jin, X.; Liu, X. Energy-Efficient GPU Clusters Scheduling for Deep Learning. *arXiv* **2023**, arXiv:2304.06381.
105. Filippini, F.; Ardagna, D.; Lattuada, M.; Amaldi, E.; Riedl, M.; Materka, K.; Skrzypek, P.; Ciavotta, M.; Magugliani, F.; Cicala, M. ANDREAS: Artificial intelligence traiNing scheDuler foR accElerAted resource clusterS. In Proceedings of the 2021 8th International Conference on Future Internet of Things and Cloud (FiCloud), Rome, Italy, 23–25 August 2021; pp. 388–393.
106. Sun, J.; Sun, M.; Zhang, Z.; Xie, J.; Shi, Z.; Yang, Z.; Zhang, J.; Wu, F.; Wang, Z. Helios: An efficient out-of-core GNN training system on terabyte-scale graphs with in-memory performance. *arXiv* **2023**, arXiv:2310.00837.
107. Zhou, Y.; Zeng, W.; Zheng, Q.; Liu, Z.; Chen, J. A Survey on Task Scheduling of CPU-GPU Heterogeneous Cluster. *ZTE Commun.* **2024**, *22*, 83.
108. Zhang, H.; Stafman, L.; Or, A.; Freedman, M.J. Slaq: Quality-driven scheduling for distributed machine learning. In Proceedings of the 2017 Symposium on Cloud Computing, Santa Clara, CA, USA, 25–27 September 2017; pp. 390–404.
109. Narayanan, D.; Kazhamiaka, F.; Abuzaid, F.; Kraft, P.; Agrawal, A.; Kandula, S.; Boyd, S.; Zaharia, M. Solving large-scale granular resource allocation problems efficiently with pop. In Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, Virtual, 26–29 October 2021; pp. 521–537.
110. Tumanov, A.; Zhu, T.; Park, J.W.; Kozuch, M.A.; Harchol-Balter, M.; Ganger, G.R. TetriSched: Global rescheduling with adaptive plan-ahead in dynamic heterogeneous clusters. In Proceedings of the Eleventh European Conference on Computer Systems, London, UK, 18–21 April 2016; pp. 1–16.
111. Fiat, A.; Woeginger, G.J. Competitive analysis of algorithms. In *Online Algorithms: The State of the Art*; Springer: Berlin/Heidelberg, Germany, 2005; pp. 1–12.
112. Günther, E.; Maurer, O.; Megow, N.; Wiese, A. A new approach to online scheduling: Approximating the optimal competitive ratio. In Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, New Orleans, LA, USA, 6–8 January 2013; pp. 118–128.
113. Mitzenmacher, M. Scheduling with predictions and the price of misprediction. *arXiv* **2019**, arXiv:1902.00732.
114. Han, Z.; Tan, H.; Jiang, S.H.C.; Fu, X.; Cao, W.; Lau, F.C. Scheduling placement-sensitive BSP jobs with inaccurate execution time estimation. In Proceedings of the IEEE INFOCOM 2020-IEEE Conference on Computer Communications, Virtual, 6–9 July 2020; pp. 1053–1062.
115. Mitzenmacher, M.; Shahout, R. Queueing, Predictions, and LLMs: Challenges and Open Problems. *arXiv* **2025**, arXiv:2503.07545.
116. Gao, W.; Sun, P.; Wen, Y.; Zhang, T. Titan: A scheduler for foundation model fine-tuning workloads. In Proceedings of the 13th Symposium on Cloud Computing, San Francisco, CA, USA, 8–10 November 2022; pp. 348–354.

117. Zheng, P.; Pan, R.; Khan, T.; Venkataraman, S.; Akella, A. Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning. In Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), Boston, WA, USA, 17–19 April 2023; pp. 703–723.
118. Zheng, H.; Xu, F.; Chen, L.; Zhou, Z.; Liu, F. Cynthia: Cost-efficient cloud resource provisioning for predictable distributed deep neural network training. In Proceedings of the 48th International Conference on Parallel Processing, Kyoto Japan, 5–8 August 2019; pp. 1–11.
119. Mu'alem, A.W.; Feitelson, D.G. Utilization, predictability, workloads, and user runtime estimates in scheduling the IBM SP2 with backfilling. *IEEE Trans. Parallel Distrib. Syst.* **2002**, *12*, 529–543. [CrossRef]
120. Goponenko, A.V.; Lamar, K.; Allan, B.A.; Brandt, J.M.; Dechev, D. Job Scheduling for HPC Clusters: Constraint Programming vs. Backfilling Approaches. In Proceedings of the 18th ACM International Conference on Distributed and Event-based Systems, Villeurbanne, France, 24–28 June 2024; pp. 135–146.
121. Kolker-Hicks, E.; Zhang, D.; Dai, D. A reinforcement learning based backfilling strategy for HPC batch jobs. In Proceedings of the SC'23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis, Denver, CO, USA, 12–17 November 2023; pp. 1316–1323.
122. Kwok, Y.K.; Ahmad, I. Static scheduling algorithms for allocating directed task graphs to multiprocessors. *ACM Comput. Surv. (CSUR)* **1999**, *31*, 406–471. [CrossRef]
123. Bittencourt, L.F.; Sakellariou, R.; Madeira, E.R. DAG scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm. In Proceedings of the 2010 18th Euromicro Conference on Parallel, Distributed and Network-Based Processing, Pisa, Italy, 17–19 February 2010; pp. 27–34.
124. Le, T.N.; Sun, X.; Chowdhury, M.; Liu, Z. Allox: Compute allocation in hybrid clusters. In Proceedings of the Fifteenth European Conference on Computer Systems, Heraklion, Greece, 27–30 April 2020; pp. 1–16.
125. Gu, R.; Chen, Y.; Liu, S.; Dai, H.; Chen, G.; Zhang, K.; Che, Y.; Huang, Y. Liquid: Intelligent resource estimation and network-efficient scheduling for deep learning jobs on distributed GPU clusters. *IEEE Trans. Parallel Distrib. Syst.* **2021**, *33*, 2808–2820. [CrossRef]
126. Guo, J.; Nomura, A.; Barton, R.; Zhang, H.; Matsuoka, S. Machine learning predictions for underestimation of job runtime on HPC system. In *Proceedings of the Supercomputing Frontiers: 4th Asian Conference, SCFA 2018, Singapore, 26–29 March 2018*; Proceedings 4; Springer International Publishing: Berlin/Heidelberg, Germany, 2018; pp. 179–198.
127. Mao, H.; Schwarzkopf, M.; Venkatakrishnan, S.B.; Meng, Z.; Alizadeh, M. Learning scheduling algorithms for data processing clusters. In Proceedings of the ACM Special Interest Group on Data Communication, Beijing, China, 19–23 August 2019; pp. 270–288.
128. Zhao, X.; Wu, C. Large-scale machine learning cluster scheduling via multi-agent graph reinforcement learning. *IEEE Trans. Netw. Serv. Manag.* **2021**, *19*, 4962–4974. [CrossRef]
129. Chowdhury, M.; Stoica, I. Efficient coflow scheduling without prior knowledge. *ACM SIGCOMM Comput. Commun. Rev.* **2015**, *45*, 393–406. [CrossRef]
130. Sharma, A.; Bhasi, V.M.; Singh, S.; Kesidis, G.; Kandemir, M.T.; Das, C.R. GPU cluster scheduling for network-sensitive deep learning. *arXiv* **2024**, arXiv:2401.16492.
131. Gu, D.; Zhao, Y.; Zhong, Y.; Xiong, Y.; Han, Z.; Cheng, P.; Yang, F.; Huang, G.; Jin, X.; Liu, X. ElasticFlow: An elastic serverless training platform for distributed deep learning. In Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Vancouver, BC, Canada, 25–29 March 2023; Volume 2, pp. 266–280.
132. Even, G.; Halldórsson, M.M.; Kaplan, L.; Ron, D. Scheduling with conflicts: Online and offline algorithms. *J. Sched.* **2009**, *12*, 199–224. [CrossRef]
133. Diaz, C.O.; Pecero, J.E.; Bouvry, P. Scalable, low complexity, and fast greedy scheduling heuristics for highly heterogeneous distributed computing systems. *J. Supercomput.* **2014**, *67*, 837–853. [CrossRef]
134. Wei, J.; He, J.; Chen, K.; Zhou, Y.; Tang, Z. Collaborative filtering and deep learning based recommendation system for cold start items. *Expert Syst. Appl.* **2017**, *69*, 29–39. [CrossRef]
135. Wu, Y.; Ma, K.; Yan, X.; Liu, Z.; Cai, Z.; Huang, Y.; Cheng, J.; Yuan, H.; Yu, F. Elastic deep learning in multi-tenant GPU clusters. *IEEE Trans. Parallel Distrib. Syst.* **2021**, *33*, 144–158. [CrossRef]
136. Shukla, D.; Sivathanu, M.; Viswanatha, S.; Gulavani, B.; Nehme, R.; Agrawal, A.; Chen, C.; Kwatra, N.; Ramjee, R.; Sharma, P.; et al. Singularity: Planet-scale, preemptive and elastic scheduling of AI workloads. *arXiv* **2022**, arXiv:2202.07848.
137. Saxena, V.; Jayaram, K.; Basu, S.; Sabharwal, Y.; Verma, A. Effective elastic scaling of deep learning workloads. In Proceedings of the 2020 28th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), Nice, France, 17–19 November 2020; pp. 1–8.
138. Gujarati, A.; Karimi, R.; Alzayat, S.; Hao, W.; Kaufmann, A.; Vigfusson, Y.; Mace, J. Serving DNNs like clockwork: Performance predictability from the bottom up. In Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), Online, 4–6 November 2020; pp. 443–462.

139. Wang, H.; Liu, Z.; Shen, H. Job scheduling for large-scale machine learning clusters. In Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies, Barcelona, Spain, 1–4 December 2020; pp. 108–120.
140. Schrage, L. A proof of the optimality of the shortest remaining processing time discipline. *Oper. Res.* **1968**, *16*, 687–690. [CrossRef]
141. Hwang, C.; Kim, T.; Kim, S.; Shin, J.; Park, K. Elastic resource sharing for distributed deep learning. In Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21), Online, 12–14 April 2021; pp. 721–739.
142. Graham, R.L. Combinatorial scheduling theory. In *Mathematics Today Twelve Informal Essays*; Springer: Berlin/Heidelberg, Germany, 1978; pp. 183–211.
143. Han, J.; Rafique, M.M.; Xu, L.; Butt, A.R.; Lim, S.H.; Vazhkudai, S.S. Marble: A multi-GPU aware job scheduler for deep learning on HPC systems. In Proceedings of the 2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID), Melbourne, Australia, 11–14 May 2020; pp. 272–281.
144. Baptiste, P. Polynomial time algorithms for minimizing the weighted number of late jobs on a single machine with equal processing times. *J. Sched.* **1999**, *2*, 245–252. [CrossRef]
145. Liu, C.L.; Layland, J.W. Scheduling algorithms for multiprogramming in a hard-real-time environment. *J. ACM (JACM)* **1973**, *20*, 46–61. [CrossRef]
146. Bao, Y.; Peng, Y.; Wu, C.; Li, Z. Online job scheduling in distributed machine learning clusters. In Proceedings of the IEEE INFOCOM 2018-IEEE Conference on Computer Communications, Honolulu, HI, USA, 15–19 April 2018; pp. 495–503.
147. Garey, M.R.; Johnson, D.S.; Sethi, R. The complexity of flowshop and jobshop scheduling. *Math. Oper. Res.* **1976**, *1*, 117–129. [CrossRef]
148. Graham, R.L. Bounds for certain multiprocessing anomalies. *Bell Syst. Tech. J.* **1966**, *45*, 1563–1581. [CrossRef]
149. Deng, X.; Liu, H.N.; Long, J.; Xiao, B. Competitive analysis of network load balancing. *J. Parallel Distrib. Comput.* **1997**, *40*, 162–172. [CrossRef]
150. Zhou, R.; Pang, J.; Zhang, Q.; Wu, C.; Jiao, L.; Zhong, Y.; Li, Z. Online scheduling algorithm for heterogeneous distributed machine learning jobs. *IEEE Trans. Cloud Comput.* **2022**, *11*, 1514–1529. [CrossRef]
151. Memeti, S.; Pllana, S.; Binotto, A.; Kołodziej, J.; Brandic, I. Using meta-heuristics and machine learning for software optimization of parallel computing systems: A systematic literature review. *Computing* **2019**, *101*, 893–936. [CrossRef]
152. Yoo, A.B.; Jette, M.A.; Grondona, M. Slurm: Simple Linux utility for resource management. In Proceedings of the Workshop on Job Scheduling Strategies for Parallel Processing, Seattle, WA, USA, 24 June 2003; Springer: Berlin/Heidelberg, Germany, 2003; pp. 44–60.
153. Scully, Z.; Grosof, I.; Harchol-Balter, M. Optimal multiserver scheduling with unknown job sizes in heavy traffic. *ACM SIGMETRICS Perform. Eval. Rev.* **2020**, *48*, 33–35. [CrossRef]
154. Rai, I.A.; Urvoy-Keller, G.; Biersack, E.W. Analysis of LAS scheduling for job size distributions with high variance. In Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, Pittsburgh, PA, USA, 17–21 June 2003; pp. 218–228.
155. Sultana, A.; Chen, L.; Xu, F.; Yuan, X. E-LAS: Design and analysis of completion-time agnostic scheduling for distributed deep learning cluster. In Proceedings of the 49th International Conference on Parallel Processing, Edmonton, AB, Canada, 17–20 August 2020; pp. 1–11.
156. Menear, K.; Nag, A.; Perr-Sauer, J.; Lunacek, M.; Potter, K.; Duplyakin, D. Mastering HPC runtime prediction: From observing patterns to a methodological approach. In Proceedings of the Practice and Experience in Advanced Research Computing 2023: Computing for the Common Good, Portland, OR, USA, 23–27 July 2023; pp. 75–85.
157. Luan, Y.; Chen, X.; Zhao, H.; Yang, Z.; Dai, Y. SCHED^2: Scheduling Deep Learning Training via Deep Reinforcement Learning. In Proceedings of the 2019 IEEE Global Communications Conference (GLOBECOM), Big Island, HI, USA, 9–13 December 2019; pp. 1–7.
158. Qin, H.; Zawad, S.; Zhou, Y.; Yang, L.; Zhao, D.; Yan, F. Swift machine learning model serving scheduling: A region based reinforcement learning approach. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, Denver, CO, USA, 17–22 November 2019; pp. 1–23.
159. Peng, Y.; Bao, Y.; Chen, Y.; Wu, C.; Meng, C.; Lin, W. DL2: A deep learning-driven scheduler for deep learning clusters. *IEEE Trans. Parallel Distrib. Syst.* **2021**, *32*, 1947–1960. [CrossRef]
160. Chen, Z.; Quan, W.; Wen, M.; Fang, J.; Yu, J.; Zhang, C.; Luo, L. Deep learning research and development platform: Characterizing and scheduling with QoS guarantees on GPU clusters. *IEEE Trans. Parallel Distrib. Syst.* **2019**, *31*, 34–50. [CrossRef]
161. Kim, S.; Kim, Y. Co-scheML: Interference-aware container co-scheduling scheme using machine learning application profiles for GPU clusters. In Proceedings of the 2020 IEEE International Conference on Cluster Computing (CLUSTER), Kobe, Japan, 14–17 September 2020; pp. 104–108.
162. Duan, J.; Song, Z.; Miao, X.; Xi, X.; Lin, D.; Xu, H.; Zhang, M.; Jia, Z. Parcae: Proactive, Liveput-Optimized DNN Training on Preemptible Instances. In Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), Santa Clara, CA, USA, 16–18 April 2024; pp. 1121–1139.

163. Yi, X.; Zhang, S.; Luo, Z.; Long, G.; Diao, L.; Wu, C.; Zheng, Z.; Yang, J.; Lin, W. Optimizing distributed training deployment in heterogeneous GPU clusters. In Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies, Barcelona, Spain, 1–4 December 2020; pp. 93–107.
164. Ryu, J.; Eo, J. Network contention-aware cluster scheduling with reinforcement learning. In Proceedings of the 2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS), Danzhou, China, 17–21 December 2023; pp. 2742–2745.
165. Fan, Y.; Lan, Z.; Childers, T.; Rich, P.; Allcock, W.; Papka, M.E. Deep reinforcement agent for scheduling in HPC. In Proceedings of the 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS), Virtual, 17–21 May 2021; pp. 807–816.
166. Hu, Q.; Zhang, M.; Sun, P.; Wen, Y.; Zhang, T. Lucid: A non-intrusive, scalable and interpretable scheduler for deep learning training jobs. In Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Vancouver, BC, Canada, 25–29 March 2023; Volume 2, pp. 457–472.
167. Zhou, P.; He, X.; Luo, S.; Yu, H.; Sun, G. JPAS: Job-progress-aware flow scheduling for deep learning clusters. *J. Netw. Comput. Appl.* **2020**, *158*, 102590. [CrossRef]
168. Xiao, W.; Ren, S.; Li, Y.; Zhang, Y.; Hou, P.; Li, Z.; Feng, Y.; Lin, W.; Jia, Y. AntMan: Dynamic scaling on GPU clusters for deep learning. In Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), Online, 4–6 November 2020; pp. 533–548.
169. Xie, L.; Zhai, J.; Wu, B.; Wang, Y.; Zhang, X.; Sun, P.; Yan, S. Elan: Towards generic and efficient elastic training for deep learning. In Proceedings of the 2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS), Singapore, 29 November–1 December 2020; pp. 78–88.
170. Ding, J.; Ma, S.; Dong, L.; Zhang, X.; Huang, S.; Wang, W.; Zheng, N.; Wei, F. Longnet: Scaling transformers to 1,000,000,000 tokens. *arXiv* **2023**, arXiv:2307.02486.
171. Liu, J.; Wu, Z.; Feng, D.; Zhang, M.; Wu, X.; Yao, X.; Yu, D.; Ma, Y.; Zhao, F.; Dou, D. Heterps: Distributed deep learning with reinforcement learning based scheduling in heterogeneous environments. *Future Gener. Comput. Syst.* **2023**, *148*, 106–117. [CrossRef]
172. Chiang, M.C.; Chou, J. DynamoML: Dynamic Resource Management Operators for Machine Learning Workloads. In Proceedings of the CLOSER, Virtual, 28–30 April 2021; pp. 122–132.
173. Li, J.; Xu, H.; Zhu, Y.; Liu, Z.; Guo, C.; Wang, C. Lyra: Elastic scheduling for deep learning clusters. In Proceedings of the Eighteenth European Conference on Computer Systems, Rome, Italy, 8–12 May 2023; pp. 835–850.
174. Albahar, H.; Dongare, S.; Du, Y.; Zhao, N.; Paul, A.K.; Butt, A.R. Schedtune: A heterogeneity-aware GPU scheduler for deep learning. In Proceedings of the 2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid), Taormina, Italy, 16–19 May 2022; pp. 695–705.
175. Robertsson, J.O.; Blanch, J.O.; Nihei, K.; Tromp, J. *Numerical Modeling of Seismic Wave Propagation: Gridded Two-Way Wave-Equation Methods*; Society of Exploration Geophysicists: Houston, TX, USA, 2012.
176. Bég, O.A. Numerical methods for multi-physical magnetohydrodynamics. *J. Magnetohydrodyn. Plasma Res.* **2013**, *18*, 93.
177. Yang, J.; Liu, T.; Tang, G.; Hu, T. Modeling seismic wave propagation within complex structures. *Appl. Geophys.* **2009**, *6*, 30–41. [CrossRef]
178. Koch, S.; Weiland, T. Time domain methods for slowly varying fields. In Proceedings of the 2010 URSI International Symposium on Electromagnetic Theory, Berlin, Germany, 16–19 August 2010; pp. 291–294.
179. Christodoulou, D.; Miao, S. *Compressible Flow and Euler's Equations*; International Press: Somerville, MA, USA, 2014; Volume 9.
180. Guillet, T.; Pakmor, R.; Springel, V.; Chandrashekar, P.; Klingenberg, C. High-order magnetohydrodynamics for astrophysics with an adaptive mesh refinement discontinuous Galerkin scheme. *Mon. Not. R. Astron. Soc.* **2019**, *485*, 4209–4246. [CrossRef]
181. Caddy, R.V.; Schneider, E.E. Cholla-MHD: An exascale-capable magnetohydrodynamic extension to the cholla astrophysical simulation code. *Astrophys. J.* **2024**, *970*, 44. [CrossRef]
182. Müller, E.H.; Scheichl, R.; Vainikko, E. Petascale elliptic solvers for anisotropic PDEs on GPU clusters. *arXiv* **2014**, arXiv:1402.3545.
183. Xue, W.; Roy, C.J. Multi-GPU performance optimization of a CFD code using OpenACC on different platforms. *arXiv* **2020**, arXiv:2006.02602.
184. Mariani, G.; Anghel, A.; Jongerius, R.; Dittmann, G. Predicting cloud performance for hpc applications: A user-oriented approach. In Proceedings of the 2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID), Madrid, Spain, 14–17 May 2017; pp. 524–533.
185. Chan, C.P.; Bachan, J.D.; Kenny, J.P.; Wilke, J.J.; Beckner, V.E.; Almgren, A.S.; Bell, J.B. Topology-aware performance optimization and modeling of adaptive mesh refinement codes for exascale. In Proceedings of the 2016 First International Workshop on Communication Optimizations in HPC (COMHPC), Salt Lake City, UT, USA, 16–18 November 2016; pp. 17–28.
186. Bender, M.A.; Bunde, D.P.; Demaine, E.D.; Fekete, S.P.; Leung, V.J.; Meijer, H.; Phillips, C.A. Communication-aware processor allocation for supercomputers. In Proceedings of the Workshop on Algorithms and Data Structures, Waterloo, ON, Canada, 15–17 August 2005; Springer: Berlin/Heidelberg, Germany, 2005; pp. 169–181.

187. Calore, E.; Gabbana, A.; Schifano, S.F.; Tripiccione, R. Evaluation of DVFS techniques on modern HPC processors and accelerators for energy-aware applications. *Concurr. Comput. Pract. Exp.* **2017**, *29*, e4143. [CrossRef]
188. Narayanan, D.; Santhanam, K.; Phanishayee, A.; Zaharia, M. Accelerating deep learning workloads through efficient multi-model execution. In Proceedings of the NeurIPS Workshop on Systems for Machine Learning, Montreal, QC, Canada, 8 December 2018;Volume 20.
189. Jayaram, K.; Muthusamy, V.; Dube, P.; Ishakian, V.; Wang, C.; Herta, B.; Boag, S.; Arroyo, D.; Tantawi, A.; Verma, A.; et al. FfDL: A flexible multi-tenant deep learning platform. In Proceedings of the 20th International Middleware Conference, Davis, CA, USA, 9–13 December 2019; pp. 82–95.
190. Narayanan, D.; Santhanam, K.; Kazhamiaka, F.; Phanishayee, A.; Zaharia, M. Analysis and exploitation of dynamic pricing in the public cloud for ML training. In Proceedings of the VLDB DISPA Workshop 2020, Online, 31 August–4 September 2020.
191. Wang, S.; Gonzalez, O.J.; Zhou, X.; Williams, T.; Friedman, B.D.; Havemann, M.; Woo, T. An efficient and non-intrusive GPU scheduling framework for deep learning training systems. In Proceedings of the SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, Atlanta, GA, USA, 9–19 November 2020; pp. 1–13.
192. Yu, P.; Chowdhury, M. Fine-grained GPU sharing primitives for deep learning applications. *Proc. Mach. Learn. Syst.* **2020**, *2*, 98–111.
193. Yang, Z.; Ye, Z.; Fu, T.; Luo, J.; Wei, X.; Luo, Y.; Wang, X.; Wang, Z.; Zhang, T. Tear up the bubble boom: Lessons learned from a deep learning research and development cluster. In Proceedings of the 2022 IEEE 40th International Conference on Computer Design (ICCD), Olympic Valley, CA, USA, 23–26 October 2022; pp. 672–680.
194. Cui, W.; Zhao, H.; Chen, Q.; Zheng, N.; Leng, J.; Zhao, J.; Song, Z.; Ma, T.; Yang, Y.; Li, C.; et al. Enable simultaneous DNN services based on deterministic operator overlap and precise latency prediction. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, St. Louis, MO, USA, 14–19 November 2021; pp. 1–15.
195. Zhao, H.; Han, Z.; Yang, Z.; Zhang, Q.; Yang, F.; Zhou, L.; Yang, M.; Lau, F.C.; Wang, Y.; Xiong, Y.; et al. HiveD: Sharing a GPU cluster for deep learning with guarantees. In Proceedings of the 14th USENIX symposium on operating systems design and implementation (OSDI 20), Online, 4–6 November 2020; pp. 515–532.
196. Jeon, M.; Venkataraman, S.; Qian, J.; Phanishayee, A.; Xiao, W.; Yang, F. *Multi-Tenant GPU Clusters for Deep Learning Workloads: Analysis and Implications*; Technical Report; Microsoft Research: Redmond, WA, USA, 2018.
197. Li, W.; Chen, S.; Li, K.; Qi, H.; Xu, R.; Zhang, S. Efficient online scheduling for coflow-aware machine learning clusters. *IEEE Trans. Cloud Comput.* **2020**, *10*, 2564–2579. [CrossRef]
198. Dutta, S.B.; Naghibijouybari, H.; Gupta, A.; Abu-Ghazaleh, N.; Marquez, A.; Barker, K. Spy in the GPU-box: Covert and side channel attacks on multi-GPU systems. In Proceedings of the 50th Annual International Symposium on Computer Architecture, Orlando, FL, USA, 17–21 June 2023; pp. 1–13.
199. Wang, W.; Ma, S.; Li, B.; Li, B. Coflex: Navigating the fairness-efficiency tradeoff for coflow scheduling. In Proceedings of the IEEE INFOCOM 2017-IEEE Conference on Computer Communications, Atlanta, GA, USA, 1–4 May 2017; pp. 1–9.
200. Li, Z.; Shen, H. Co-Scheduler: A coflow-aware data-parallel job scheduler in hybrid electrical/optical datacenter networks. *IEEE/ACM Trans. Netw.* **2022**, *30*, 1599–1612. [CrossRef]
201. Pavlidakis, M.; Vasiliadis, G.; Mavridis, S.; Argyros, A.; Chazapis, A.; Bilas, A. Guardian: Safe GPU Sharing in Multi-Tenant Environments. In Proceedings of the 25th International Middleware Conference, Hong Kong, China, 2–6 December 2024; pp. 313–326.
202. Zhao, W.; Jayarajan, A.; Pekhimenko, G. Tally: Non-Intrusive Performance Isolation for Concurrent Deep Learning Workloads. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Rotterdam, The Netherlands, 30 March–3 April 2025; Volume 1, pp. 1052–1068.
203. Xue, C.; Cui, W.; Zhao, H.; Chen, Q.; Zhang, S.; Yang, P.; Yang, J.; Li, S.; Guo, M. A codesign of scheduling and parallelization for large model training in heterogeneous clusters. *arXiv* **2024**, arXiv:2403.16125.
204. Zheng, L.; Li, Z.; Zhang, H.; Zhuang, Y.; Chen, Z.; Huang, Y.; Wang, Y.; Xu, Y.; Zhuo, D.; Xing, E.P.; et al. Alpa: Automating inter-and Intra-Operator parallelism for distributed deep learning. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), Carlsbad, CA, USA, 1–13 July 2022; pp. 559–578.
205. Athlur, S.; Saran, N.; Sivathanu, M.; Ramjee, R.; Kwatra, N. Varuna: Scalable, low-cost training of massive deep learning models. In Proceedings of the Seventeenth European Conference on Computer Systems, Rennes, France, 5–8 April 2022; pp. 472–487.
206. Ousterhout, K.; Wendell, P.; Zaharia, M.; Stoica, I. Sparrow: Distributed, low latency scheduling. In Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles, Farmington, PA USA, 3–6 November 2013; pp. 69–84.
207. Yuan, B.; He, Y.; Davis, J.; Zhang, T.; Dao, T.; Chen, B.; Liang, P.S.; Re, C.; Zhang, C. Decentralized training of foundation models in heterogeneous environments. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 25464–25477.
208. Sun, B.; Huang, Z.; Zhao, H.; Xiao, W.; Zhang, X.; Li, Y.; Lin, W. Llumnix: Dynamic scheduling for large language model serving. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), Santa Clara CA USA, 10–12 July 2024; pp. 173–191.

209. Jiang, Z.; Lin, H.; Zhong, Y.; Huang, Q.; Chen, Y.; Zhang, Z.; Peng, Y.; Li, X.; Xie, C.; Nong, S.; et al. {MegaScale}: Scaling large language model training to more than 10,000 GPUs. In Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), Santa Clara, CA, USA, 16–18 April 2024; pp. 745–760.
210. Shahout, R.; Malach, E.; Liu, C.; Jiang, W.; Yu, M.; Mitzenmacher, M. Don't Stop Me Now: Embedding-based Scheduling for LLMs. *arXiv* **2024**, arXiv:2410.01035.
211. Mei, Y.; Zhuang, Y.; Miao, X.; Yang, J.; Jia, Z.; Vinayak, R. Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. *arXiv* **2024**, arXiv:2406.01566.
212. Mitzenmacher, M.; Vassilvitskii, S. Algorithms with predictions. *Commun. ACM* **2022**, *65*, 33–35. [CrossRef]
213. Li, Y.; Phanishayee, A.; Murray, D.; Tarnawski, J.; Kim, N.S. Harmony: Overcoming the hurdles of GPU memory capacity to train massive DNN models on commodity servers. *arXiv* **2022**, arXiv:2202.01306. [CrossRef]
214. Ye, Z.; Gao, W.; Hu, Q.; Sun, P.; Wang, X.; Luo, Y.; Zhang, T.; Wen, Y. Deep learning workload scheduling in G datacenters: A survey. *ACM Comput. Surv.* **2024**, *56*, 1–38. [CrossRef]
215. Chang, Z.; Xiao, S.; He, S.; Yang, S.; Pan, Z.; Li, D. Frenzy: A Memory-Aware Serverless LLM Training System for Heterogeneous GPU Clusters. *arXiv* **2024**, arXiv:2412.14479.
216. Moussaid, A. Investigating the Impact of Prompt Engineering Techniques on Energy Consumption in Large Language Models. Master's Thesis, University of L'Aquila, L'Aquila, Italy, 2025.
217. Yao, C.; Liu, W.; Tang, W.; Hu, S. EAIS: Energy-aware adaptive scheduling for CNN inference on high-performance GPUs. *Future Gener. Comput. Syst.* **2022**, *130*, 253–268. [CrossRef]
218. Khan, O.; Yu, J.; Kim, Y.; Seo, E. Efficient Adaptive Batching of DNN Inference Services for Improved Latency. In Proceedings of the 2024 International Conference on Information Networking (ICOIN), Ho Chi Minh City, Vietnam, 17–19 January 2024; pp. 197–200.
219. Beltrán, E.T.M.; Pérez, M.Q.; Sánchez, P.M.S.; Bernal, S.L.; Bovet, G.; Pérez, M.G.; Pérez, G.M.; Celdrán, A.H. Decentralized federated learning: Fundamentals, state of the art, frameworks, trends, and challenges. *IEEE Commun. Surv. Tutor.* **2023**, *25*, 2983–3013. [CrossRef]
220. Bharadwaj, S.; Das, S.; Mazumdar, K.; Beckmann, B.M.; Kosonocky, S. Predict; don't react for enabling efficient fine-grain DVFS in GPUs. In Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Vancouver, BC, Canada, 25–29 March 2023; Volume 4, pp. 253–267.
221. Albers, S. Energy-efficient algorithms. *Commun. ACM* **2010**, *53*, 86–96. [CrossRef]
222. Han, Y.; Nan, Z.; Zhou, S.; Niu, Z. DVFS-Aware DNN Inference on GPUs: Latency Modeling and Performance Analysis. *arXiv* **2025**, arXiv:2502.06295.
223. Kakolyris, A.K.; Masouros, D.; Vavaroutsos, P.; Xydis, S.; Soudris, D. SLO-aware GPU Frequency Scaling for Energy Efficient LLM Inference Serving. *arXiv* **2024**, arXiv:2408.05235.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.